
Gradient-Based Causal Tree Ensembles: A Backbone Architecture for Heterogeneous Treatment Effects

Yusuke Kano¹ Jeremy Paul Voisey² Mihaela van der Schaar³

Abstract

Estimating Heterogeneous Treatment Effects (HTE) from observational data is essential in fields such as healthcare and policy-making, where randomized experiments are often impractical. While representation learning-based methods have shown promise, recent studies suggest that tree-based approaches may offer superior performance on tabular data, particularly in the presence of uninformative features. We introduce **GR**Adient-based Causal tree **E**nsembles (GRACE), a novel tree-based architecture for HTE estimation that incorporates multi-way, oblique, and soft splits, enabling end-to-end training via backpropagation. GRACE can be seamlessly integrated into existing models as a replacement for fully-connected neural network layers. Across diverse benchmarks involving binary and non-binary treatment settings, GRACE consistently surpasses neural network and tree-based baselines, often by a substantial margin. We further analyze GRACE as an extension of fully-connected neural network layers and conduct ablation studies to isolate and quantify the contribution of each architectural component to the improvement in performance. These results position GRACE as a promising new foundation for flexible, robust, and accurate HTE estimation. GRACE code is available at <https://github.com/ysk-kano/GRACE>

1. Introduction

Estimating treatment effects from observational data is a critical task in domains such as healthcare and policy-making, where randomized controlled trials are often infeasible or unethical. With the rapid advancement of machine learning techniques, a wide range of methods for Heterogeneous Treatment Effect (HTE) estimation have been proposed (Bica et al., 2021). In particular, representation learning-based approaches have gained significant attention in recent years (see Li et al. (2024) for a review). However, recent findings reported in Curth et al. (2021) suggest that tree-based methods, such as Causal Forests (Wager & Athey, 2018), may outperform neural network-based models depending on the underlying data-generating process. This observation aligns with trends in supervised learning on tabular data, where tree-based models often demonstrate superior performance (Shwartz-Ziv & Armon, 2021; Uddin & Lu, 2024). One reason for this is that neural networks struggle with uninformative features, which are common in real-world tabular datasets (Grinsztajn et al., 2022). Motivated by these insights, we hypothesize that tree-based methods generally yield more accurate HTE estimates with observational tabular data.

To test this hypothesis, we introduce **GR**Adient-based Causal tree **E**nsembles (GRACE), a novel tree-based architecture for HTE estimation, which can be executed within modern ML frameworks such as PyTorch (Paszke et al., 2019). Notably, our method is designed to be modular and can be seamlessly substituted for a fully-connected neural network (FCNN) layer, allowing flexible integration into existing architectures for HTE estimation. An overview of a GRACE layer and stacked architecture for HTE estimation is given in Figure 1. Inspired by the recent advancement in differentiable tree-based models for supervised learning (Yang et al., 2018; Popov et al., 2019; Marton et al., 2024), GRACE consists of multiple trees with multi-way, oblique, and soft splits, that are fully parameterized and can be trained end-to-end by backpropagation. Through extensive experiments in binary and more complex treatment settings, we demonstrate that GRACE consistently outperforms baseline methods across a wide range of scenarios. Furthermore, we analyze a GRACE layer as an extension

¹Canon Inc., Tokyo, Japan ²Canon Medical Research Europe, Edinburgh, United Kingdom ³Department of Applied Mathematics and Theoretical Physics, University of Cambridge, Cambridge, United Kingdom. Correspondence to: Yusuke Kano <yusuke.kano@mre.medical.canon>.

of an FCNN layer, comparing notation and carrying out an ablation analysis to isolate the contribution of each architectural component to the improvement in performance.

Our contributions are summarized as follows:

1. We propose a novel gradient-based tree ensemble method for HTE estimation, compatible with modern ML frameworks (e.g. PyTorch) that is easily interchangeable with a FCNN layer, enabling flexible integration into existing architectures.
2. We provide a detailed analysis of the relationship between our proposed method and a FCNN layer, decomposing their differences into key components to quantify each one’s contribution to the improvement in performance.
3. We conduct extensive experiments on multiple benchmark datasets across binary and more complex treatment settings, empirically validating the effectiveness of our approach and demonstrating superior performance in diverse scenarios.

Conflict of Interest Disclosure. The author YK is employed by Canon Inc., and JPV is employed by Canon Medical Research Europe. These organizations are involved in the development of the methods presented in this paper.

2. Related Work

We briefly review prior work that is relevant to our method.

Deep learning-based method. Most deep learning-based methods aim to learn representations that capture complex treatment-outcome relationships and mitigate covariate shift. TARNet and CFRNet (Shalit et al., 2017) introduced architectures that utilize shared and treatment-specific representations. Building on this foundation, DR-CFR (Hassanpour & Greiner, 2019), DeR-CFR (Wu et al., 2021) and SNet (Curth & Schaar, 2021) proposed a decomposition strategy to enhance robustness by disentangling representation and outcome modeling. Beyond binary treatment settings, several recent works have explored more complex treatment settings (see Wang et al. (2026) for a review). For bundle treatment setting that involve large treatment spaces, NCoRE (Parbhoo et al., 2021) extends the TARNet architecture, while RMNet (Tanimoto et al., 2021) and VSR (Zou et al., 2020) focus on extracting treatment representations. In contrast, SCP (Qian et al., 2021) introduced a data augmentation approach to address the sparsity in the observed treatment assignments. For continuous treatment settings, VCNet (Nie et al., 2020) introduces a neural network architecture based on varying-coefficient models to capture dose-response relationships. More recently, TransTEE (Zhang et al., 2023) applied transformer-based architectures to causal inference,

demonstrating their applicability across various treatment scenarios.

Tree-based method. Many tree-based approaches are developed by adapting existing tree models for causal inference. Causal Forests (Wager & Athey, 2018) extend random forests to estimate HTE by leveraging local neighborhoods and sub-sampling strategies. Bayesian Additive Regression Trees (BART) (Chipman et al., 2010; Hill, 2011) and Bayesian Causal Forests (BCF) (Hahn et al., 2020) offer a Bayesian nonparametric alternative, modeling HTE with uncertainty quantification and flexible function approximation.

Differentiable trees for supervised learning. Differentiable tree architectures have emerged in supervised learning as a promising direction for combining the strengths of trees for tabular data with the optimization benefits of neural networks. Deep Neural Decision Trees (Yang et al., 2018) introduced multi-way, soft decision boundaries that allow gradient-based training. NODE (Popov et al., 2019) proposed a fully differentiable ensemble of oblivious decision trees and multi-layer architectures. Extending this line of work, GRANDE (Marton et al., 2023) incorporated axis-aligned decision trees and a dense representation of tree structures to support non-oblivious decision trees, enhancing both expressiveness and scalability.

While our work builds upon these differentiable tree-based models, it introduces several important contributions beyond prior methods such as GRANDE (Marton et al., 2023). In particular, our approach differs from existing differentiable decision-tree architectures by generalizing the dense representation to support multi-way splits, thereby enabling a more expressive structure without increasing depth. Furthermore, we address a key limitation of existing differentiable tree models, namely their high computational cost, by introducing a novel cost-reduction strategy that improves the efficiency of the routing mechanism while preserving model expressiveness.

3. Problem Formalization

3.1. Heterogeneous Treatment Effects (HTE)

We assume N observed samples $(x_i, t_i^f, y_i^f)_{i=1}^N$, which are independent realizations of the random variables $(\mathbf{X}, \mathbf{T}^f, Y^f)$. Here, $\mathbf{X}$ denotes a pre-treatment feature vector in some feature space $\mathcal{X}$, $\mathbf{T}^f$ represents the factual treatment assignment within the set of all possible treatments $\mathcal{T}$, and Y^f is the observed outcome, which can be either continuous or binary. The treatment variable $\mathbf{T}^f$ varies depending on the settings: $\{0, 1\}$ for binary treatment, $\{0, 1, \dots, m\}$ for multi-valued treatment, $\mathbb{R}$ for continuous treatment, $\{0, 1\}^m$ for bundle treatment, and $\mathbb{R}^m$ for multi-dimensional treatment.

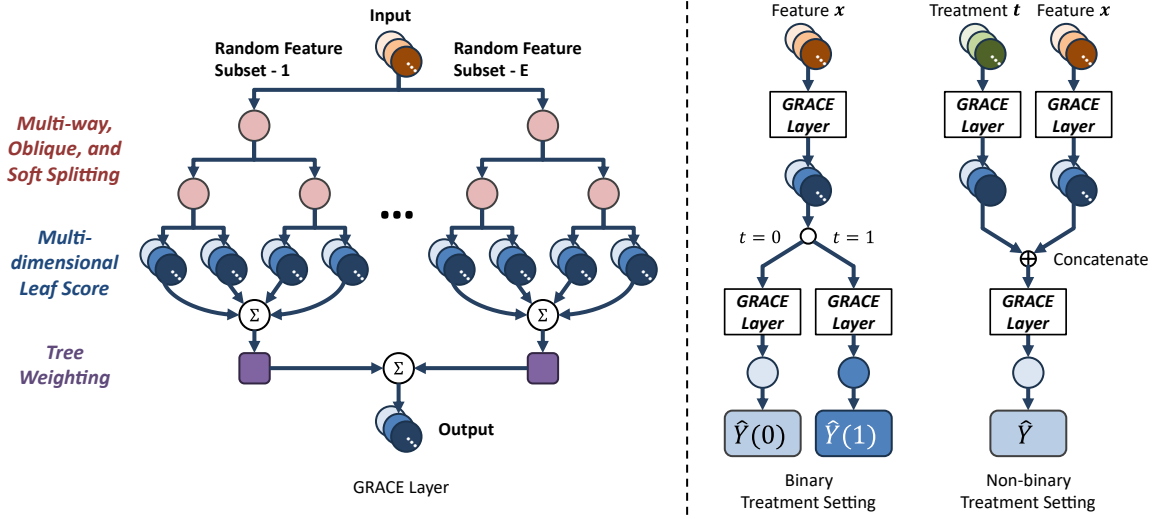


Figure 1. Overview of GRACE architecture

Following Neyman-Rubin potential outcomes framework (Rubin, 2005), we assume a potential outcome $Y(T = t)$ for each possible treatment $t \in \mathcal{T}$. The goal is to estimate the potential outcomes for a given set of input features:

$$\mu(t, x) = \mathbb{E}(Y(t) | X = x) \quad (1)$$

We refer to $\mu(\cdot)$ as a Conditional Treatment Response Function (CTRF)¹. Specifically for binary treatment, the Conditional Average Treatment Effect (CATE) $\tau(\cdot)$ is defined as the difference between the two expected potential outcomes:

$$\tau(x) = \mathbb{E}(Y(1) - Y(0) | X = x) \quad (2)$$

We seek to estimate CTRF and CATE under the following identifiability assumptions.

Assumption 1 (Unconfoundedness): There are no unobserved confounders, such that the potential outcomes $Y(t)$ are independent of the treatment T , given the covariates X , i.e. $Y(t) \perp\!\!\!\perp T^f | X$.

Assumption 2 (Overlap): The probability of assignment to a particular treatment is positive and non-deterministic, i.e. $0 < \mathbb{P}(t|x) < 1$.

3.2. Objectives and Performance Metrics

One of the primary objectives in HTE estimation is to minimize the estimation error. In the binary treatment setting, the root mean squared error (RMSE) of the CATE, defined

¹This generalizes Conditional Dose Response Function (CDRF), which is commonly used for continuous treatment settings.

as follows, can serve as a performance metric:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\tau(x_i) - \hat{\tau}(x_i))^2} \quad (3)$$

This is also known as the Precision in Estimation of Heterogeneous Effect (PEHE) (Hill, 2011), which is widely used in literature. For multi-valued and bundle treatment setting, we adopt the following metric (Yoon et al., 2018; Parbhoo et al., 2021):

$$\text{RMSE} = \sqrt{\frac{1}{N} \frac{1}{|\mathcal{T}|} \sum_{i=1}^N \sum_{t_i \in \mathcal{T}} (\mu(x_i, t_i) - \hat{\mu}(x_i, t_i))^2} \quad (4)$$

For continuous and multi-dimensional treatment settings, we define an analogous metric:

$$\text{RMSE} = \sqrt{\frac{1}{N} \frac{1}{K} \sum_{i=1}^N \sum_{k=1}^K (\mu(x_i, t_i^k) - \hat{\mu}(x_i, t_i^k))^2} \quad (5)$$

where t_i^k denotes the k -th randomly sampled treatment from the treatment space $\mathcal{T}$.

Although RMSE serves as our primary evaluation metric, it may not fully capture all aspects of model performance (Curth et al., 2021). In particular, metrics based solely on error may not adequately reflect the quality of treatment effect rankings or the utility of the model for downstream decision making. To address this limitation, we additionally consider decision-oriented metrics such as Regret, AUC, and

Kendall’s tau, which provide complementary perspectives on model behavior. A detailed discussion of these metrics is provided in Appendix A.

4. Gradient-based Causal Tree Ensembles

We propose **GR**Adient-based **C**ausal tree **E**nsembles (GRACE), a novel tree-based model for HTE estimation using ensembles and gradient descent optimization technique. In short, the model comprises multiple trees with multi-way, oblique, and soft splits, which are fully parameterized, and are trained end-to-end using backpropagation.

4.1. GRACE Architecture

A GRACE layer takes a multi-dimensional input and maps it to a multi-dimensional output, similar to a FCNN layer. The architecture design is guided by three key motivations.

Firstly, we adopt multi-way, oblique, and soft splitting strategies to enhance the representation of a single tree. This contrasts with traditional tree-based approaches, which typically rely on binary, axis-aligned, and hard splits often requiring deeper and more complex trees to achieve comparable expressiveness. **Secondly**, each leaf node generates a multi-dimensional output, which can represent potential outcomes or serve as learned representations for downstream tasks. This design allows us to capitalize on recent advancements in representation learning for HTE estimation by replacing a FCNN layer with a GRACE layer. **Finally**, the tree is fully differentiable, enabling its parameters to be optimized end-to-end using backpropagation. This addresses a major limitation of conventional tree-based approaches (e.g. Causal Forest (Wager & Athey, 2018)), which rely on greedy optimization and therefore cannot always identify the globally optimal splits for the entire tree.

Compared to FCNN layers, GRACE exhibits two key differences that are particularly important for tabular HTE estimation.

First, GRACE promotes sparse feature selection, effectively filtering out uninformative covariates that FCNN layers often struggle to ignore due to their dense connectivity. However, sparsity alone may reduce the expressive capacity and smoothness of individual decision paths. GRACE addresses this limitation by combining multi-way, oblique, and soft splitting with an ensemble strategy, thereby restoring smoothness and enhancing representational power while retaining the benefits of sparsity. **Second**, GRACE naturally captures complex feature interactions, including higher-order interactions between treatment variables and covariates. In FCNN layers, such multiplicative interactions must be approximated through deep compositions of nonlinear transformations, often requiring many layers. In contrast, GRACE leverages a tree-based routing mechanism that in-

herently encodes multiplicative structure through feature-dependent branching. Consequently, even a single GRACE layer can efficiently represent high-order interactions that would otherwise require significantly deeper FCNN architectures.

These differences are further analyzed in detail in Section 5.1, where we provide a more theoretical comparison.

4.2. Formulation of a Single Tree

Next, we describe a single decision tree architecture, which is the core building block of our GRACE layer. In terms of arithmetic formulation, we adopt a fully grown decision tree architecture and dense representation (Marton et al., 2024) to enable efficient computation in the tensor format.

Consider a vector $\mathbf{x} \in \mathbb{R}^k$ containing k features as inputs to a fully-grown decision tree of depth d with b -way splits. Without loss of generality, we assume that the features are numeric, as categorical features can be handled by encoding them as numeric vectors. For all branches at an internal node, a oblique decision boundary function $\mathbb{O} : \mathbb{R}^k \rightarrow \mathbb{R}^{b-1} \in [0, 1]^{b-1}$ is defined with the following form:

$$\mathbb{O}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\iota}, \boldsymbol{\tau}) = \sigma \left(\boldsymbol{\mu} \odot \left(\mathbb{1}_{b-1} \sum_{i=1}^k c_i \mathbf{x}_i - \boldsymbol{\tau} \right) \right) \quad (6)$$

with $\mathbf{c} = \text{softsign}(\boldsymbol{\iota}) \odot \text{entmax}(|\boldsymbol{\iota}|)$,

where $\boldsymbol{\iota} \in \mathbb{R}^k$ is a vector for the feature selection, $\boldsymbol{\tau} \in \mathbb{R}^{b-1}$ is a vector for thresholds, $\boldsymbol{\mu} \in \mathbb{R}^{b-1}$ is a trainable scaling vector, $\odot$ is an element-wise product, $\sigma(\cdot)$ is a sigmoid function, and $\mathbb{1}_n$ is an n -dimensional vector in which all the values are 1. Entmax (Peters et al., 2019) is expected to generate a sparse selection of features, which plays an important role in filtering out uninformative features. The elements of the thresholds $\boldsymbol{\tau}$ are ordered to be monotonically increasing, by sorting them in each forward pass.

In our soft splitting approach, each branch is associated with a continuous value representing its split weight. Unlike conventional decision-tree routing probabilities, these weights are not necessarily normalized and therefore do not generally sum to 1 within a node. To obtain these weights, oblique decision boundaries are transformed by a soft split function $\mathbb{S} : \mathbb{R}^k \rightarrow \mathbb{R}^b \in [0, 1]^b$, where $\mathbb{S}_i$ (the function for a branch i) can be defined with the following form:

$$\mathbb{S}_i(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\iota}, \boldsymbol{\tau}) = \prod_{j=1}^{b-1} \left(p_{ij} (1 - \mathbb{O}(\mathbf{x})_j) + (1 - p_{ij}) \mathbb{O}(\mathbf{x})_j \right)$$

with $p_{ij} = \begin{cases} 0 & \text{if } i > j \\ 1 & \text{otherwise.} \end{cases}$

(7)

For example, for the 3-way split, the split weight is calculated as $\mathbb{S} = [(1 - \mathbb{O}_1)(1 - \mathbb{O}_2), \mathbb{O}_1(1 - \mathbb{O}_2), \mathbb{O}_1\mathbb{O}_2]^T$.

We now extend the vectors $\boldsymbol{\iota} \in \mathbb{R}^k$, $\boldsymbol{\tau} \in \mathbb{R}^{b-1}$, $\boldsymbol{\mu} \in \mathbb{R}^{b-1}$ to matrices $\mathbf{I} \in \mathbb{R}^{v \times k}$, $\mathbf{T} \in \mathbb{R}^{v \times (b-1)}$, $\mathbf{M} \in \mathbb{R}^{v \times (b-1)}$ to represent all the internal nodes, where $v = \frac{b^d - 1}{b - 1}$ corresponds to the number of internal nodes. It should be noted that the internal nodes of a fully-grown tree are enumerated in a breadth-first order, starting from index 1. The membership degree for a specific leaf node can be calculated by tracing all the outputs of the split functions that lead to that leaf node. We now define the membership degree function $\mathbb{L} : \mathbb{R}^k \rightarrow \mathbb{R}^{b^d} \in [0, 1]^{b^d}$, where $\mathbb{L}_i$ (the function for a leaf i) can be expressed with the following form:

$$\mathbb{L}_i(\mathbf{x}; \mathbf{M}, \mathbf{I}, \mathbf{T}) = \prod_{j=1}^d \mathbb{S}(\mathbf{x}; \mathbf{M}, \mathbf{I}, \mathbf{T})_{\alpha(i,j), \beta(i,j)} \quad (8)$$

with

$$\alpha(i, j) = \frac{b^{j-1} - 1}{b - 1} + \left\lfloor \frac{i - 1}{b^{d-(j-1)}} \right\rfloor + 1$$

$$\beta(i, j) = \left(\left\lfloor \frac{i - 1}{b^{d-j}} \right\rfloor \bmod b \right) + 1.$$

For example, for the tree with depth 4 and 3-way split, the membership degree for leaf node 12 is calculated as $\mathbb{L}_{12} = \mathbb{S}_{1,1} \cdot \mathbb{S}_{2,2} \cdot \mathbb{S}_{6,1} \cdot \mathbb{S}_{17,3}$, which means the output aggregates the split weights for the branches [1, 2, 1, 3] at the internal nodes [1, 2, 6, 17].

Finally, a trainable score vector is assigned in each leaf node, which is multiplied by the membership degree to calculate the output for each leaf node. These score vectors for all the leaf nodes can be described in a matrix form including a bias term $\mathbf{L} \in \mathbb{R}^{h \times (b^d + 1)}$, where h is a dimension of output vectors. Combining the formulations discussed above, a single tree acts as a function $g : \mathbb{R}^k \rightarrow \mathbb{R}^h$ to generate a output score vector, based on its membership degree $\mathbb{L}$ and leaf score $\mathbf{L}$.

$$g(\mathbf{x}; \mathbf{M}, \mathbf{I}, \mathbf{T}, \mathbf{L}) = \mathbf{L} \cdot [\mathbb{L}(\mathbf{x}; \mathbf{M}, \mathbf{I}, \mathbf{T}), 1]^T \quad (9)$$

4.3. From a Single Tree to Tree Ensembles

For tree ensembles, we take the weighted average of the multiple trees. Consider tree ensembles consisting of E trees. By introducing the weights $\boldsymbol{\omega} \in \mathbb{R}^E$ and 3D tensors $\mathbf{M}, \mathbf{I}, \mathbf{T}, \mathbf{L}$ extended from M, I, T, L respectively for all tree parameters, tree ensembles can be defined as a function $\mathbb{G} : \mathbb{R}^k \rightarrow \mathbb{R}^h$:

$$\mathbb{G}(\mathbf{x}; \boldsymbol{\omega}, \mathbf{M}, \mathbf{I}, \mathbf{T}, \mathbf{L}) = \sum_{e=1}^E \omega_e g(\mathbf{x}; \mathbf{M}_e, \mathbf{I}_e, \mathbf{T}_e, \mathbf{L}_e) \quad (10)$$

In a similar way to Random Forest, a randomly selected subset of the features with a predefined ratio ρ is used in a

single tree. Furthermore, we applied dropout at tree level by randomly deactivating the elements of the output score vector with a predefined ratio λ .

4.4. Multi-layer Stacking

In view of that the tree ensemble $\mathbb{G} : \mathbb{R}^k \rightarrow \mathbb{R}^h$ is a function mapping from feature space into a different vector space, it can also be viewed as a way of learning representations for the HTE estimation, where the dimension of output scores h corresponds to a dimension of representation. One way to take advantage of this aspect is to stack the tree ensembles in a similar way to multi-layer neural networks. Another advantage of learning representation is to be able to apply any existing representation learning based approach for the architecture and regularization, including disentangled representation learning (such as DR-CFR (Hassanpour & Greiner, 2019), SNet (Curth & Schaar, 2021)) and minimizing the distribution discrepancy (such as CFRNet (Shalit et al., 2017), ESCFR (Wang et al., 2023)).

For binary treatment setting, we adopt a TARNet-inspired approach, where layers dedicated to each treatment are followed by layers shared across treatments, as illustrated in Figure 1. For a 2-layer GRACE configuration, the output can be expressed as:

$$\hat{y}(t) = (1 - t)\mathbb{G}_{Y0}(\mathbb{G}_s(\mathbf{x})) + t\mathbb{G}_{Y1}(\mathbb{G}_s(\mathbf{x})) \quad (11)$$

where $\mathbb{G}_s$ denotes the shared layer and $\mathbb{G}_{Y0}, \mathbb{G}_{Y1}$ are treatment-specific layers.

For non-binary treatment settings, the feature vector and treatment vector are embedded and concatenated before being passed to three subsequent layer, as shown in Figure 1. For a 2-layer GRACE configuration, the output is given by:

$$\hat{y}(t) = \mathbb{G}_Y([\mathbb{G}_X(\mathbf{x}) || \mathbb{G}_T(t)]) \quad (12)$$

where $[\cdot || \cdot]$ denotes the vector concatenation, $\mathbb{G}_X$ and $\mathbb{G}_T$ represent the embedding layers, and $\mathbb{G}_Y$ is the output layer.

In terms of training objective, we primarily focus on the factual loss shown below.

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N L(\hat{y}_i(t_i^f), y_i^f), \quad (13)$$

where $L(\cdot)$ refers to the squared loss if Y is continuous and binary cross-entropy if Y is binary. While this can be extended to incorporate additional losses as discussed above, we do not explore this aspect in present work. More detailed implementation of GRACE can be found in Appendix B.

5. Analysis

5.1. Comparison with a Fully-connected layer

In this section, we compare GRACE with a FCNN layer to clarify the differences between them. We begin with Theorem 5.1 (with a proof in Appendix C.1) to highlight the key components of GRACE.

Theorem 5.1. *For the GRACE layer, there exists a sparse tensor $\tilde{\mathbf{W}} \in \mathbb{R}^{E \times I \times J \times k}$ such that the layer can be expressed as follows:*

$$\mathbb{G}(\mathbf{x}; \omega, \tilde{\mathbf{W}}, \mathbf{L}) = \sum_{e=1}^E \sum_{i=1}^I \omega_e \mathbf{L}_{e,i,:} \prod_{j=1}^J f(\tilde{\mathbf{W}}_{e,i,:,j} \mathbf{x})_j \quad (14)$$

where $I = b^d$, $J = (b-1)d$, $f(\cdot)$ denotes a nonlinear activation function. For simplicity, and with a slight abuse of notation, we omit the bias terms. This indicates that a GRACE layer is considered as an extension of FCNN, in the sense that it equips 1) an ensemble approach, 2) sparse feature selection, and 3) tree routing with products between features. This finding also allows us to consider different variants between FCNN and GRACE, with the typical variants described in the Table 1. We use these variants in our ablation study to identify how important each element is.

5.2. GRACE-R: Relaxed form for efficient computation

The primary limitation of the GRACE layer lies in the exponential growth of both the number of parameters and computational cost with respect to the depth and the number of branches of decision trees, with complexity scaling as $\mathcal{O}(b^d)$. To address this issue, we propose a relaxed version of GRACE layer that mitigates these scaling challenges.

The first step involves introducing an oblivious tree structure, in which the split parameters are shared across all nodes at the same depth. Each element of the relaxed membership degree function $\tilde{\mathbb{L}}_i$ can be transformed using the relaxed soft split function $\tilde{\mathbb{S}} : \mathbb{R}^k \rightarrow \mathbb{R}^{d \times b}$, as follows:

$$\begin{aligned} \tilde{\mathbb{L}}_i(\mathbf{x}) &= \prod_{j=1}^d \tilde{\mathbb{S}}(\mathbf{x})_{j,\beta(i,j)} \\ \text{with } \beta(i,j) &= \left(\left\lfloor \frac{i-1}{b^{d-j}} \right\rfloor \bmod b \right) + 1 \end{aligned} \quad (15)$$

This design reduces the number of parameters for the split function to grow linearly with respect to both tree depth and branching factor. However, since the number of leaf nodes remains unchanged, both the number of parameters and the computational cost are still dominated by $\mathcal{O}(b^d)$.

To further alleviate this, the second step introduces parameter sharing across the score vectors associated with leaf

nodes. We assume that each row vector $\mathbf{L}_{i,:}$ can be decomposed into a set of sub-vectors $\mathbf{l}^{(j,\beta(i,j))}$, each corresponding to the branch taken at depth j that leads to leaf i . In other words, the decomposition satisfies $\mathbf{L}_{i,:} = \prod_{j=1}^d \mathbf{l}^{(j,\beta(i,j))}$. This yields the following form of the relaxed tree function $\tilde{g} : \mathbb{R}^k \rightarrow \mathbb{R}^h$ (see derivation in Appendix C.2):

$$\begin{aligned} \tilde{g}(\mathbf{x}) &= \sum_{i=1}^{b^d} \mathbf{L}_{i,:} \prod_{j=1}^d \tilde{\mathbb{S}}(\mathbf{x})_{j,\beta(i,j)} \\ &= \prod_{j=1}^d \left(\sum_{a_j=1}^b \mathbf{l}^{(j,a_j)} \cdot \tilde{\mathbb{S}}(\mathbf{x})_{j,a_j} \right) \end{aligned} \quad (16)$$

Consequently, the model achieves linear complexity $\mathcal{O}(bd)$, and we refer to this relaxed formulation as GRACE-R. A comparison of the actual training times is provided in Appendix D.

6. Empirical Evaluations

Throughout the experiments, we compare GRACE with several baseline methods, including both deep learning-based and tree-based approaches, using RMSE (Eq. 3, 4, or 5 with $K = 20$) on test samples. Hyper-parameters are tuned for each dataset using factual loss on validation data, and the same tuned parameters are applied across all training sample sizes.

Details of the data generating processes are provided in Appendix E, baseline implementation in Appendix F, hyper-parameter tuning in Appendix G, hyper-parameter sensitivity analysis in Appendix H, and full results including additional metrics in Appendix I.

6.1. Toy Dataset

Firstly, we use a simple proof-of-concept ‘‘toy’’ dataset inspired by Wager & Athey (2018) to demonstrate the model’s performance and robustness to the uninformative features.

Setup. A binary treatment is randomly assigned to each sample. Input features are drawn from uniform distribution, with the feature dimension k set to 5, 20, or 100. However, only the first two features contribute to the treatment effect. We generate 5,000 samples for training and 10,000 samples for testing. We compare GRACE with Causal Forest (CF) (Wager & Athey, 2018) and TARNet (Shalit et al., 2017), reporting averages over 100 simulations.

Result. Table 2 shows a comparison of the average RMSE for GRACE and the baselines. GRACE outperformed the baselines for all feature dimensions. Notably, the performance did not degrade even when $k = 100$. Figure 2 illustrates the comparison of τ -surfaces between the true treatment effect $\tau(\mathbf{x})$ and estimates $\hat{\tau}(\mathbf{x})$ produced by GRACE,

Table 1. Comparison between FCNN, GRACE, and their variants. E, S, T correspond to Ensemble, Sparse feature selection, and Tree routing. $\mathbf{h}^{(l)}$ denotes the output of l -th layer, and $f^{(l)}(\cdot)$ represents a nonlinear activation function at the l -th layer. $\mathbf{L}^{(l)}$, $\mathbf{W}^{(l)}$ are weight matrices, $\mathbf{L}^{(l)}$, $\mathbf{W}^{(l)}$ are weight tensors, and $\tilde{\mathbf{W}}^{(l)}$ is a sparse weight tensor.

Name	E	S	T	Expression
FCNN				$\mathbf{h}^{(l)} = f^{(l)}(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)})$
FCNN _E	✓			$\mathbf{h}^{(l)} = \sum_{e=1}^E \omega_e^{(l)} f^{(l)}(\mathbf{W}_{e,:}^{(l)} \mathbf{h}^{(l-1)})$
FCNN _{ES}	✓	✓		$\mathbf{h}^{(l)} = \sum_{e=1}^E \omega_e^{(l)} f^{(l)}(\tilde{\mathbf{W}}_{e,:}^{(l)} \mathbf{h}^{(l-1)})$
FCNN _{ET}	✓		✓	$\mathbf{h}^{(l)} = \sum_{e=1}^E \sum_{i=1}^I \omega_{e,i}^{(l)} \mathbf{L}_{e,i,:}^{(l)} \prod_{j=1}^J f^{(l)}(\mathbf{W}_{e,i,j}^{(l)} \mathbf{h}^{(l-1)})_j$
FCNN _{ST}		✓	✓	$\mathbf{h}^{(l)} = \sum_{i=1}^I \mathbf{L}_{i,:}^{(l)} \prod_{j=1}^J f^{(l)}(\tilde{\mathbf{W}}_{i,j}^{(l)} \mathbf{h}^{(l-1)})_j$
GRACE	✓	✓	✓	$\mathbf{h}^{(l)} = \sum_{e=1}^E \sum_{i=1}^I \omega_{e,i}^{(l)} \mathbf{L}_{e,i,:}^{(l)} \prod_{j=1}^J f^{(l)}(\tilde{\mathbf{W}}_{e,i,j}^{(l)} \mathbf{h}^{(l-1)})_j$

Causal Forest and TARNet for different feature dimensions ($k = 5, 20$, or 100). GRACE accurately capture true surfaces, regardless of feature dimensions. This also highlights that Causal Forest and TARNet make different types of error, which are not seen in GRACE. Causal Forest may struggle to capture the smooth surfaces due to the nature of axis-aligned splitting, whereas TARNet may struggle with axis-aligned surfaces because it is difficult for neural networks to completely ignore the uninformative features.

Table 2. RMSE averaged over 100 simulations, standard error in parentheses.

k	GRACE	CF	TARNet
5	0.024 ± 0.000	0.041 ± 0.000	0.034 ± 0.000
20	0.026 ± 0.000	0.046 ± 0.000	0.057 ± 0.000
100	0.024 ± 0.000	0.051 ± 0.000	0.135 ± 0.007

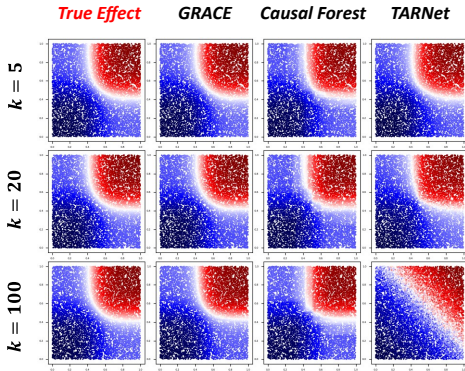


Figure 2. Comparison of τ -surface. The test points are plotted according to their first two coordinates (the only ones that contribute to treatment effect). The treatment effect is denoted by color, from blue (low) to red (high).

6.2. Synthetic Dataset

Next, we conduct an evaluation with the synthetic datasets to demonstrate the performance across different underlying data generating processes and training sample sizes.

Setup. For the binary treatment setting, we consider 3 synthetic data generating processes, in which $Y(0)$, $Y(1)$, and $\mathbb{P}(t)$ are simulated based on (i) combinations of first-, second-, and third-order feature interactions (Interaction-based DGP), (ii) a FCNN with three shared layers and three dedicated layers (FCNN-based DGP), and (iii) an axis-aligned decision tree with depth 10 (Tree-based DGP). For the non-binary treatment setting, We extend the (i) Interaction-based DGP to (iv) multi-valued, (v) continuous, and (vi) bundle treatment settings by concatenating treatment vector with feature vector. In all settings, $k = 25$ dimensional features are sampled from a normal distribution $X \sim N(0, 1)$. Each experiment is evaluated on 1,000 independently generated test samples, while the training sample size is varied across 500, 1,000, 5,000, and 10,000. We include all baseline methods listed in Appendix F, and report averages over 100 simulation runs.

Result. Figure 3a and 3b show RMSE of each method across different size of training data. GRACE outperformed the baselines for almost all data sizes and across all experimental settings. This results indicate that GRACE can robustly estimate the HTE across a wide range of data generating processes, even when the training data size is small. Furthermore, the performance gap was particularly large in scenarios involving high-order feature interactions. This suggests that the GRACE architecture naturally captures complex feature interactions, a point we further investigate in the ablation study in Section 6.4.

6.3. Semi-synthetic Dataset

Finally, we extend our experiments to semi-synthetic datasets, including commonly used datasets such as IHDP (Hill, 2011) and ACIC 2016 (Dorie et al., 2019) for the binary treatment setting, as well as TCGA and News (Schwab et al., 2020; Bica et al., 2020) for the non-binary treatment setting. TCGA and News additionally feature high-dimensional input features ($> 2,000$), providing a challenging evaluation setting. We also introduce the Glucose dataset, generated using an FDA-approved simulator

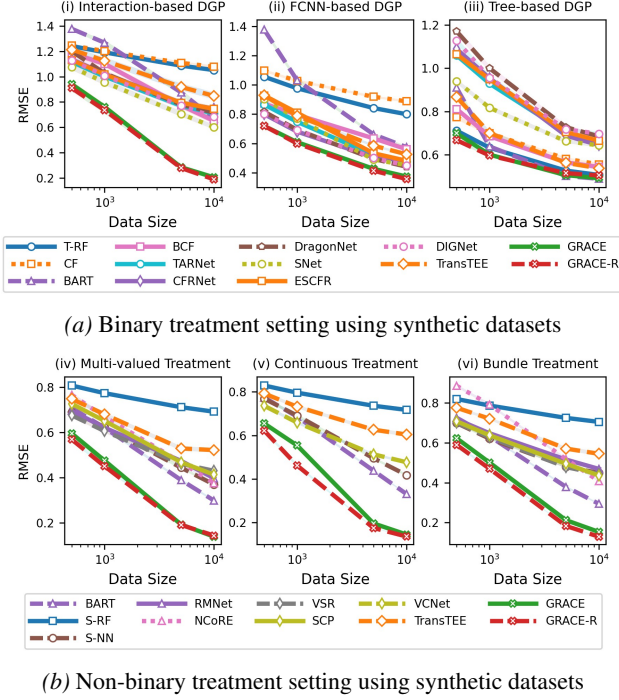


Figure 3. Performance comparison using **synthetic datasets**. The shaded areas indicate standard error.

(Esponera & Cinà, 2025) for Type-1 diabetes, which provides a more realistic response surface.

Setup for IHDP, ACIC 2016, TCGA, and News. For IHDP, TCGA and News, we ran the training and testing repeatedly with 100 simulations using different random seeds. For ACIC 2016, we ran a single training and testing process per setting for each of the 75 settings. Model performance is evaluated on test samples, which were split from the dataset using a 10% holdout ratio. Training sample sizes were varied across 100, 200, 500, 1,000, and the full remaining 90% of the dataset (i.e. after the 10% test holdout) for IHDP and ACIC 2016. For TCGA and News, the training sizes were 500, 1,000, 5,000 and the full remaining 90% . We include all baseline methods listed in Appendix F, and report the averages over all simulations.

Setup for Glucose. The task is to predict the glucose level one hour after the observation, given a specified insulin administration. We consider both binary and multi-dimensional treatment settings. In the binary treatment setting, treatment is whether basal insulin is administered or not. In the multi-dimensional treatment setting, we additionally incorporate meal intake and bolus insulin administration, resulting in a four-dimensional treatment consisting of meal timing, meal amount, bolus volume, and basal volume. To emulate real-world conditions, glucose values are obtained from a sensor that includes measurement noise.

Model performance is evaluated on 500 test samples, while the training sample size is varied across 100, 200, 500, and 1,000. We include all baseline methods listed in Appendix F, and report the averages over 100 simulations runs.

Results. Figure 4a and 4b show RMSE for each method across different size of training data. The observed trends are consistent with those reported in the synthetic data experiments, providing strong evidence that GRACE can robustly estimate HTE across diverse scenarios, including settings with high-dimensional input features and more realistic response surfaces.

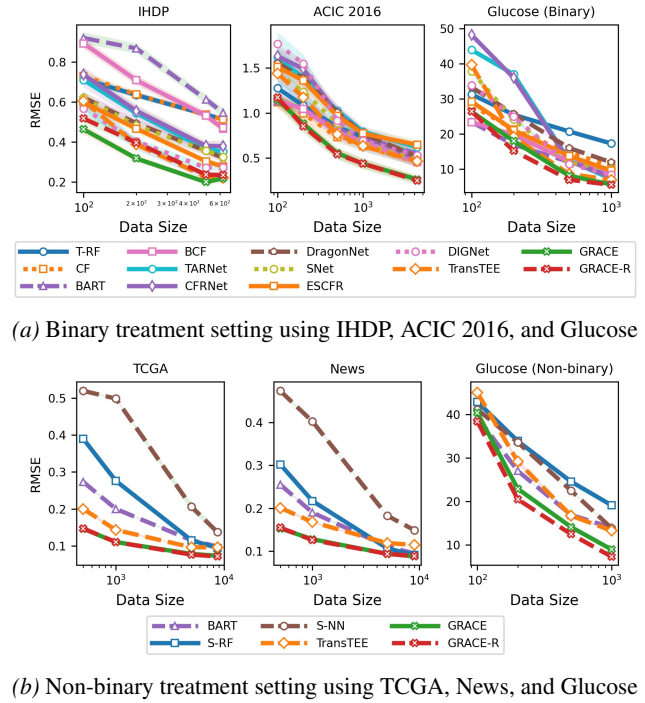


Figure 4. Performance comparison using **semi-synthetic datasets**. The shaded areas indicate standard error.

6.4. Ablation Study

Setup. For the comparison of layer variants, we fixed the architecture to a TARNet-like structure, with treatment-specific layers followed by shared layers across treatments. All layers are then replaced with those specified in Section 5.1. We used the same experimental setup as in the 3 settings for binary treatment setting described in Section 6.2.

Results. Figure 5 shows RMSE for each variant across different size of training data. As a reminder, each subscript corresponds to a specific architectural component: **E** - Ensemble, **S** - Sparse feature selection, and **T** - Tree routing. A model with FCNN layers is equivalent to TARNet. While the effectiveness of each component varies slightly depending on the setting, all elements contribute to improved pre-

dictive accuracy. Notably, the combination of **E** (ensemble approach) and **S** (sparse feature selection) is essential across all settings. In addition, **T** (tree routing) yields substantial gains in setting (i) and meaningful improvements in setting (ii). This is expected, as tree routing is effective in capturing complex feature interactions, which remain more influential in settings (i) and (ii) than in setting (iii).

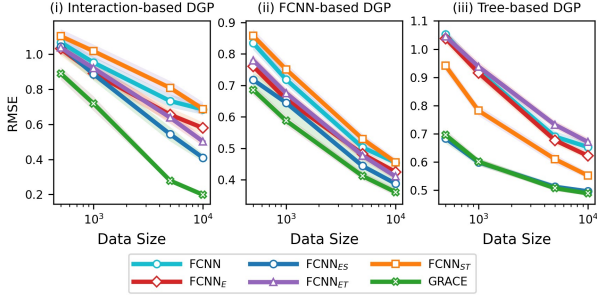


Figure 5. Performance comparison from the **ablation study**. The shaded areas indicate standard error.

7. Discussion

GRACE consistently demonstrates superior performance across synthetic and semi-synthetic settings. The results highlight three key properties of the model. First, GRACE robustly handles uninformative information and high-dimensional feature spaces, owing to its sparse feature-selection component. Second, tree-based architecture enables the model to naturally capture higher-order feature interactions. Third, the ensemble mechanism stabilizes estimation across heterogeneous local response surfaces.

Limitations & Future Work. Despite its strong performance, GRACE has several limitations. First, due to the use of ensembles, GRACE incurs higher computational costs than neural networks, even under the relaxed formulation. Second, the current implementation does not incorporate loss functions explicitly designed to mitigate covariate imbalance, which may limit its robustness in certain observational settings. Third, our experiments are restricted to static scenarios, extending GRACE to time-series treatment effect setting is a promising direction for future research.

8. Conclusion

GRACE is a new differentiable tree-based architecture for HTE estimation that integrates seamlessly with deep learning frameworks and can serve as a flexible replacement for FCNN layers. Through extensive experiments, we showed that GRACE consistently outperforms existing baselines across diverse settings. Our analysis highlights the value of sparse feature selection and tree-based structures and

suggests promising directions for future research.

Impact Statement

This work introduces GRACE, a differentiable tree-based representation layer for heterogeneous treatment effect (HTE) estimation, with applications in healthcare, public policy, and other decision-making domains. By improving modeling of treatment heterogeneity in tabular data, GRACE may enable more accurate individualized decisions, such as identifying patients most likely to benefit from a treatment or populations most affected by a policy intervention, thereby supporting more efficient resource allocation and improved outcomes.

However, its use in high-stakes settings raises important considerations. First, GRACE may inherit and amplify biases in observational data, potentially leading to inequitable outcomes. Second, although more structured than standard neural networks, it may still pose challenges for transparency and interpretability in settings requiring clear justification. Third, HTE estimates may be misinterpreted as causal without sufficient validation, potentially leading to inappropriate decisions.

Overall, while GRACE improves representational capacity for HTE estimation, its deployment requires careful attention to data quality, fairness, interpretability, and causal validity.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '19*, pp. 2623–2631, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 978-1-4503-6201-6. doi: 10.1145/3292500.3330701. URL <https://dl.acm.org/doi/10.1145/3292500.3330701>.
- Bica, I., Jordon, J., and van der Schaar, M. Estimating the Effects of Continuous-valued Interventions using Generative Adversarial Networks. In *Advances in Neural Information Processing Systems*, volume 33, pp. 16434–16445. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/bea5955b308361a1b07bc55042e25e54-Abstract.html>.
- Bica, I., Alaa, A. M., Lambert, C., and van der Schaar, M. From Real-World Patient Data to Individualized Treatment Effects Using Machine Learning: Current and Future Methods to Address Underlying Challenges. *Clinical*

- Pharmacology and Therapeutics*, 109(1):87–100, January 2021. ISSN 1532-6535. doi: 10.1002/cpt.1907.
- Brooks-Gunn, J., Liaw, F. R., and Klebanov, P. K. Effects of early intervention on cognitive function of low birth weight preterm infants. *The Journal of Pediatrics*, 120(3):350–359, March 1992. ISSN 0022-3476. doi: 10.1016/s0022-3476(05)80896-0.
- Chipman, H. A., George, E. I., and McCulloch, R. E. BART: Bayesian additive regression trees. *The Annals of Applied Statistics*, 4(1), March 2010. ISSN 1932-6157. doi: 10.1214/09-AOAS285.
- Chu, J., Sun, Z., Dong, W., Shi, J., and Huang, Z. On learning disentangled representations for individual treatment effect estimation. *Journal of Biomedical Informatics*, 124:103940, December 2021. ISSN 1532-0480. doi: 10.1016/j.jbi.2021.103940.
- Curth, A. and Schaar, M. v. d. Nonparametric Estimation of Heterogeneous Treatment Effects: From Theory to Learning Algorithms. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, pp. 1810–1818. PMLR, March 2021. URL <https://proceedings.mlr.press/v130/curth21a.html>.
- Curth, A., Svensson, D., Weatherall, J., and Schaar, M. v. d. Really Doing Great at Estimating CATE? A Critical Look at ML Benchmarking Practices in Treatment Effect Estimation. *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, August 2021. URL <https://openreview.net/forum?id=FQLzQqGEAH>.
- Dorie, V., Hill, J., Shalit, U., Scott, M., and Cervone, D. Automated versus Do-It-Yourself Methods for Causal Inference: Lessons Learned from a Data Analysis Competition. *Statistical Science*, 34(1):43–68, February 2019. ISSN 0883-4237, 2168-8745. doi: 10.1214/18-STS667. URL <https://projecteuclid.org/journals/statistical-science/volume-34/issue-1/Automated-versus-Do-It-Yourself-Methods-for-Causal-Inference/10.1214/18-STS667.full>.
- Esponera, A. and Cinà, G. Inducing Causal Structure Applied to Glucose Prediction for T1DM Patients. In *Proceedings of the Fourth Conference on Causal Learning and Reasoning*, pp. 920–946. PMLR, June 2025. URL <https://proceedings.mlr.press/v275/esponera25a.html>.
- Fernández-Loría, C. and Provost, F. Causal Decision Making and Causal Effect Estimation Are Not the Same... and Why It Matters. *INFORMS Journal on Data Science*, 1(1):4–16, April 2022. ISSN 2694-4022. doi: 10.1287/ijds.2021.0006. URL <https://pubsonline.informs.org/doi/abs/10.1287/ijds.2021.0006>.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on typical tabular data? *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, June 2022. URL https://openreview.net/forum?id=Fp7__phQszn.
- Hahn, P. R., Murray, J. S., and Carvalho, C. M. Bayesian Regression Tree Models for Causal Inference: Regularization, Confounding, and Heterogeneous Effects (with Discussion). *Bayesian Analysis*, 15(3):965–1056, September 2020. ISSN 1936-0975, 1931-6690. doi: 10.1214/19-BA1195. URL <https://projecteuclid.org/journals/bayesian-analysis/volume-15/issue-3/Bayesian-Regression-Tree-Model-s-for-Causal-Inference--Regularization-Confounding/10.1214/19-BA1195.full>.
- Hassanpour, N. and Greiner, R. Learning Disentangled Representations for Counterfactual Regression. *International Conference on Learning Representations*, September 2019. URL <https://openreview.net/forum?id=HkxBJT4YvB>.
- Hill, J. Bayesian Nonparametric Modeling for Causal Inference. *Journal of Computational and Graphical Statistics*, 20:217–240, March 2011. doi: 10.1198/jcgs.2010.08162.
- Huang, Y., Siyi, W., Leung, C. H., Wu, Q., Wang, D., and Huang, Z. DIGNet: Learning Decomposed Patterns in Representation Balancing for Treatment Effect Estimation. *Transactions on Machine Learning Research*, January 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=Z20FInfWlm>.
- Izmailov, P., Podoprikin, D., Garipov, T., Vetrov, D., and Wilson, A. G. Averaging Weights Leads to Wider Optima and Better Generalization. *arXiv preprint*, February 2019. doi: 10.48550/arXiv.1803.05407. URL <http://arxiv.org/abs/1803.05407>.
- Künzel, S. R., Sekhon, J. S., Bickel, P. J., and Yu, B. Metalearners for estimating heterogeneous treatment effects using machine learning. In *Proceedings of the National Academy of Sciences*, volume 116, pp. 4156–4165. Proceedings of the National Academy of Sciences, March 2019. doi: 10.1073/pnas.1804597116. URL <https://www.pnas.org/doi/10.1073/pnas.1804597116>.
- Li, Z., Guo, X., and Qiang, S. A survey of deep causal models and their industrial applications. *Artificial Intelligence Review*, 57(11):298, September 2024. ISSN 1573-7462. doi: 10.1007/s10462-024-10886-0. URL <https://doi.org/10.1007/s10462-024-10886-0>.

- Liu, Y., Ma, X., Zhang, D., Geng, L., Wang, X., Zheng, W., and Chen, M.-H. Look Before You Leap: Systematic Evaluation of Tree-based Statistical Methods in Subgroup Identification. *Journal of biopharmaceutical statistics*, 29(6):1082–1102, 2019. ISSN 1054-3406. doi: 10.1080/10543406.2019.1584204. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC6742587/>.
- Marton, S., Lütke, S., Bartelt, C., and Stuckenschmidt, H. GRANDE: Gradient-Based Decision Tree Ensembles for Tabular Data. *The Twelfth International Conference on Learning Representations*, October 2023. URL <https://openreview.net/forum?id=XEFWBxi075>.
- Marton, S., Lütke, S., Bartelt, C., and Stuckenschmidt, H. GradTree: Learning Axis-Aligned Decision Trees with Gradient Descent. *AAAI Conference on Artificial Intelligence*, 38(13):14323–14331, March 2024. ISSN 2374-3468. doi: 10.1609/aaai.v38i13.29345. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29345>.
- Nie, L., Ye, M., Liu, Q., and Nicolae, D. VCNet and Functional Targeted Regularization For Learning Causal Effects of Continuous Treatments. *International Conference on Learning Representations*, October 2020. URL <https://openreview.net/forum?id=RmB-88r9dL>.
- Parbhoo, S., Bauer, S., and Schwab, P. NCoRE: Neural Counterfactual Representation Learning for Combinations of Treatments. *arXiv preprint*, March 2021. doi: 10.48550/arXiv.2103.11175. URL <http://arxiv.org/abs/2103.11175>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: an imperative style, high-performance deep learning library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, number 721, pp. 8026–8037, Red Hook, NY, USA, 2019. Curran Associates Inc. URL <https://dl.acm.org/doi/10.5555/3454287.3455008>.
- Peters, B., Niculae, V., and Martins, A. F. T. Sparse Sequence-to-Sequence Models. In Korhonen, A., Traum, D., and Márquez, L. (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 1504–1519, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1146. URL <https://aclanthology.org/P19-1146/>.
- Popov, S., Morozov, S., and Babenko, A. Neural Oblivious Decision Ensembles for Deep Learning on Tabular Data. *International Conference on Learning Representations*, September 2019. URL <https://openreview.net/forum?id=rleiu2VtwH>.
- Qian, Z., Curth, A., and Schaar, M. v. d. Estimating Multi-cause Treatment Effects via Single-cause Perturbation. *Advances in Neural Information Processing Systems*, November 2021. URL <https://openreview.net/forum?id=oz3tlBrfNO>.
- Rubin, D. B. Causal Inference Using Potential Outcomes: Design, Modeling, Decisions. *Journal of the American Statistical Association*, 100(469):322–331, March 2005. ISSN 0162-1459. doi: 10.1198/016214504000001880. URL <https://doi.org/10.1198/016214504000001880>.
- Schwab, P., Linhardt, L., Bauer, S., Buhmann, J. M., and Karlen, W. Learning Counterfactual Representations for Estimating Individual Dose-Response Curves. *AAAI Conference on Artificial Intelligence*, 34(04):5612–5619, April 2020. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v34i04.6014.
- Shalit, U., Johansson, F. D., and Sontag, D. Estimating individual treatment effect: generalization bounds and algorithms. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML’17*, pp. 3076–3085, Sydney, NSW, Australia, 2017. JMLR.org. URL <https://dl.acm.org/doi/10.5555/3305890.3305999>.
- Shi, C., Blei, D. M., and Veitch, V. Adapting neural networks for the estimation of treatment effects. In *33rd International Conference on Neural Information Processing Systems*, number 225, pp. 2507–2517, Red Hook, NY, USA, 2019. Curran Associates Inc. URL <https://dl.acm.org/doi/10.5555/3454287.3454512>.
- Shwartz-Ziv, R. and Armon, A. Tabular Data: Deep Learning is Not All You Need. *8th ICML Workshop on Automated Machine Learning (AutoML)*, July 2021. URL <https://openreview.net/forum?id=vdgt epSlpV>.
- Tanimoto, A., Sakai, T., Takenouchi, T., and Kashima, H. Regret Minimization for Causal Inference on Large Treatment Space. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, pp. 946–954. PMLR, March 2021. URL <https://proceedings.mlr.press/v130/tanimoto21a.html>.
- Uddin, S. and Lu, H. Confirming the statistically significant superiority of tree-based machine learning algorithms

- over their counterparts for tabular data. *PLOS ONE*, 19 (4):e0301541, April 2024. ISSN 1932-6203. doi: 10.1371/journal.pone.0301541. URL <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0301541>.
- Wager, S. and Athey, S. Estimation and Inference of Heterogeneous Treatment Effects using Random Forests. *Journal of the American Statistical Association*, 113 (523):1228–1242, July 2018. ISSN 0162-1459. doi: 10.1080/01621459.2017.1319839. URL <https://doi.org/10.1080/01621459.2017.1319839>.
- Wang, H., Fan, J., Chen, Z., Li, H., Liu, W., Liu, T., Dai, Q., Wang, Y., Dong, Z., and Tang, R. Optimal Transport for Treatment Effect Estimation. *Advances in Neural Information Processing Systems*, 36:5404–5418, December 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1160e7f31d0a74abbbelbbf7924b949c-Abstract-Conference.html.
- Wang, Y., Li, H., Zhu, M., Wu, A., Li, B., Yin, K., Xiong, R., Wu, F., and Kuang, K. Causal Inference with Complex Treatments: A Survey. *ACM Computing Surveys*, 58(9): 229:1–229:36, 2026. ISSN 0360-0300. doi: 10.1145/3789499. URL <https://dl.acm.org/doi/10.1145/3789499>.
- Wu, A., Kuang, K., Yuan, J., Li, B., Wu, R., Zhu, Q., Zhuang, Y., and Wu, F. Learning Decomposed Representation for Counterfactual Inference. *arXiv preprint*, October 2021. doi: 10.48550/arXiv.2006.07040. URL <http://arxiv.org/abs/2006.07040>.
- Yang, Y., Morillo, I. G., and Hospedales, T. M. Deep Neural Decision Trees. *arXiv preprint*, June 2018. doi: 10.48550/arXiv.1806.06988. URL <http://arxiv.org/abs/1806.06988>.
- Yoon, J., Jordon, J., and Schaar, M. v. d. GANITE: Estimation of Individualized Treatment Effects using Generative Adversarial Nets. *International Conference on Learning Representations*, February 2018. URL <https://openreview.net/forum?id=ByKWUeWA->.
- Zhang, Y., Zhang, H., Lipton, Z. C., Li, L. E., and Xing, E. Exploring Transformer Backbones for Heterogeneous Treatment Effect Estimation. *Transactions on Machine Learning Research*, April 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=1kl4YM2Q7P>.
- Zou, H., Cui, P., Li, B., Shen, Z., Ma, J., Yang, H., and He, Y. Counterfactual Prediction for Bundle Treatment. In *Advances in Neural Information Processing Systems*, volume 33, pp. 19705–19715. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/e430ad64df3de73e6be33bcb7f6d0dac-Abstract.html.

A. Additional Metrics

In this section, we discuss alternative metrics for evaluating HTE estimation, as relying solely on RMSE may not be appropriate (Curth et al., 2021). If the ultimate goal of estimating treatment effect is decision support, it is relevant to assess whether a model correctly “classifies” patients who benefit from the treatment and those are harmed by it (Curth et al., 2021). Similarly, Fernández-Loría & Provost (2022) point out that Causal Decision Making and Causal Effect Estimation can be fundamentally different in some cases: accurate causal effect estimation is not always necessary for making good intervention decisions, and more accurate estimates of causal effects do not necessarily lead to better decisions.

For decision support in binary treatment settings, the recommended treatment can be defined by converting from the estimated CATE as:

$$\hat{a}(\mathbf{x}) = \mathbb{1}\{\hat{\tau}(\mathbf{x}) > 0\} \quad (17)$$

where $\hat{\tau}(x)$ denotes the estimated CATE.

The AUC (Area Under the Receiver Operating Characteristic) has been proposed as an evaluation metric for the recommended treatment rule (Curth et al., 2021; Liu et al., 2019).

For decision support in non-binary treatment settings, we propose to evaluate model performance based on the ranking correlation among the possible treatment options. In this paper, we use Kendall’s tau to quantify this ranking consistency.

Regret can also be used as an evaluation measure both for binary and non-binary treatment settings:

$$\text{Regret}(\hat{a}) = \frac{1}{N} \sum_{i=1}^N (Y_i(a_i^*) - Y_i(\hat{a}(\mathbf{x}_i))) \quad (18)$$

where a^* is the optimal treatment that maximizes the patient outcome. Regret is minimized when $\hat{a}(\mathbf{x}_i) = a_i^*$ for all i .

For simplicity, we calculate Kendall’s tau and regret using K randomly sampled treatment from the treatment space $\mathcal{T}$ in continuous and multi-dimensional treatment settings.

The results for these metrics are reported in Appendix I.

B. Implementation of GRACE / GRACE-R

In this section, we describe the implementation details of GRACE and GRACE-R.

To handle datasets with high-dimensional inputs, we limit the number of features used in each tree of the ensemble to a maximum of 50. For optimization, similar to NODE (Popov et al., 2019) and GRANDE (Marton et al., 2023), the models are trained using the Adam optimizer without weight decay. We further apply the Stochastic Weight Averaging (SWA) technique (Izmailov et al., 2019) by averaging the model parameters over the last five consecutive checkpoints. All trainable parameters are initialized using $\mathcal{N}(0, 1)$. For the stacked architecture, we use skip connections when the number of layer exceeds 2. In the non-binary treatment setting, to match the scales of the feature and treatment representations, we apply batch normalization after concatenating them. In all experiments, we fix the number of layers to 2.

C. Detailed Derivations

C.1. Proof of Theorem 5.1

The oblique decision boundary function $\mathbb{O} : \mathbb{R}^k \rightarrow \mathbb{R}^{b-1}$, defined in Eq. 6, can be simplified using an efficient sparse matrix $\mathbf{W} \in \mathbb{R}^{(b-1) \times (k+1)}$ to yield the following transformation:

$$\begin{aligned} \mathbb{O}(\mathbf{x}) &= \sigma \left(\boldsymbol{\mu} \odot \left(\mathbb{1}_{b-1} \sum_{i=1}^k \mathbf{c}_i \mathbf{x}_i - \boldsymbol{\tau} \right) \right) \quad \left(\text{with } \mathbf{c} = \text{softsign}(\boldsymbol{\iota}) \odot \text{entmax}(|\boldsymbol{\iota}|) \right) \\ &= \sigma \begin{bmatrix} \boldsymbol{\mu}_1 (\sum_{i=1}^k \mathbf{c}_i \mathbf{x}_i - \boldsymbol{\tau}_1) \\ \boldsymbol{\mu}_2 (\sum_{i=1}^k \mathbf{c}_i \mathbf{x}_i - \boldsymbol{\tau}_2) \\ \vdots \\ \boldsymbol{\mu}_{b-1} (\sum_{i=1}^k \mathbf{c}_i \mathbf{x}_i - \boldsymbol{\tau}_{b-1}) \end{bmatrix} \\ &= \sigma(\mathbf{W}\tilde{\mathbf{x}}) \quad \left(\text{with } \mathbf{W} = [\boldsymbol{\mu}\mathbf{c}^T, -\boldsymbol{\mu} \odot \boldsymbol{\tau}], \tilde{\mathbf{x}} = [\mathbf{x}, 1]^T \right) \end{aligned}$$

We now extend the sparse matrix $\mathbf{W} \in \mathbb{R}^{(b-1) \times (k+1)}$ to a sparse tensor $\mathbf{W} \in \mathbb{R}^{v \times (b-1) \times (k+1)}$ to represent all the splits of the internal nodes, where $v = \frac{b^d-1}{b-1}$ corresponds to the number of internal nodes. This also extends the oblique decision boundary function, yielding a transformation of the form $\mathbb{O}(\mathbf{x}; \mathbf{W}) : \mathbb{R}^k \rightarrow \mathbb{R}^{v \times (b-1)}$.

Additionally, we introduce a new sparse weight tensor $\tilde{\mathbf{W}} \in \mathbb{R}^{b^d \times d(b-1) \times (k+1)}$ and a constant matrix $\tilde{\mathbf{P}} \in \mathbb{R}^{b^d \times d(b-1)}$, whose elements are chosen to satisfy the following condition.

$$\begin{aligned} \tilde{\mathbf{W}}_{i,j,:} &= \mathbf{W}_{\gamma(i,j),\delta(j),:} \\ \tilde{\mathbf{P}}_{i,j} &= \begin{cases} 0 & \text{if } \varepsilon(i,j) > \delta(i,j) \\ 1 & \text{otherwise} \end{cases} \end{aligned}$$

$$\begin{aligned} \text{with } \gamma(i,j) &= \alpha(i, \left\lfloor \frac{j}{b-1} \right\rfloor + 1), \quad \delta(j) = (j \bmod (b-1)) + 1, \quad \varepsilon(i,j) = \beta(i, \left\lfloor \frac{j}{b-1} \right\rfloor + 1) \\ \text{where } \alpha(i,j) &= \frac{b^{j-1}-1}{b-1} + \left\lfloor \frac{i-1}{b^{d-(j-1)}} \right\rfloor + 1, \quad \beta(i,j) = \left(\left\lfloor \frac{i-1}{b^{d-j}} \right\rfloor \bmod b \right) + 1 \end{aligned}$$

This process essentially entails reordering and reshaping the tensor such that its first dimension aligns with the leaf index, and its second dimension captures the depth and branching configuration. Accordingly, the oblique decision boundary function yields a transformation of the form $\mathbb{O}(\mathbf{x}; \tilde{\mathbf{W}}) : \mathbb{R}^k \rightarrow \mathbb{R}^{b^d \times d(b-1)}$. With the sparse tensor $\tilde{\mathbf{W}}$ and the matrix $\tilde{\mathbf{P}}$, the membership degree function $\mathbb{L} : \mathbb{R}^k \rightarrow \mathbb{R}^{b^d}$ and its elements defined in Eq. 8 can be simplified by the following transformation:

$$\begin{aligned} \mathbb{L}_i(\mathbf{x}) &= \prod_{j=1}^d \mathbb{S}(\mathbf{x}; \mathbf{W})_{\alpha(i,j),\beta(i,j)} \\ &= \prod_{j=1}^d \left(\prod_{k=1}^{b-1} \left(p_{\beta(i,j)k} (1 - \mathbb{O}(\mathbf{x}; \mathbf{W})_{\alpha(i,j),k}) + (1 - p_{\beta(i,j)k}) \mathbb{O}(\mathbf{x}; \mathbf{W})_{\alpha(i,j),k} \right) \right)_{\beta(i,j)} \\ &= \prod_{j=1}^{d(b-1)} \left(\tilde{\mathbf{P}}_{i,j} (1 - \mathbb{O}(\mathbf{x}; \tilde{\mathbf{W}})_{i,j}) + (1 - \tilde{\mathbf{P}}_{i,j}) \mathbb{O}(\mathbf{x}; \tilde{\mathbf{W}})_{i,j} \right) \\ &= \prod_{j=1}^{d(b-1)} \left((1 - 2\tilde{\mathbf{P}}_{i,j}) \mathbb{O}(\mathbf{x}; \tilde{\mathbf{W}})_{i,j} + \tilde{\mathbf{P}}_{i,j} \right) \\ &= \prod_{j=1}^{d(b-1)} f(\tilde{\mathbf{W}}_{i,:}, \tilde{\mathbf{x}})_j \quad \left(\text{with } f(\mathbf{z})_j = (1 - 2\tilde{\mathbf{P}}_{i,j})\sigma(\mathbf{z})_j + \tilde{\mathbf{P}}_{i,j} \right) \end{aligned}$$

Finally, we extend these transformations to tree ensembles, yielding the following final results.

$$\begin{aligned}
 \mathbb{G}(\mathbf{x}) &= \sum_{e=1}^E \omega_e g_e(\mathbf{x}) \\
 &= \sum_{e=1}^E \omega_e \mathbf{L}_{e,:,\cdot} \mathbb{L}_{e,:}(\mathbf{x}) \\
 &= \sum_{e=1}^E \sum_{i=1}^I \omega_e \mathbf{L}_{e,i,:} \prod_{j=1}^J f(\tilde{\mathbf{W}}_{e,i,:,\cdot} \tilde{\mathbf{x}})_j
 \end{aligned}$$

C.2. Derivation of relaxed formulation (Eq. 16)

The value $\beta(i, j)$, as defined in Eq. 8, corresponds to the j -th digit (from the left) of the b -ary expansion of i , when expressed using d digits. The “+1” is added to ensure 1-based indexing. Specifically, when i is written in base- b as

$$i = (a_1 - 1)b^{d-1} + (a_2 - 1)b^{d-2} + \dots + (a_d - 1)b^0 + 1$$

then $\beta(i, j)$ is equal to a_j . Consequently, $\mathbf{l}^{(j, \beta(i, j))}$ and $\tilde{\mathbf{S}}(\mathbf{x})_{j, \beta(i, j)}$ can be rewritten as $\mathbf{l}^{(j, a_j)}$ and $\tilde{\mathbf{S}}(\mathbf{x})_{j, a_j}$, respectively. Summing over all i is therefore equivalent to summing independently over each a_j . Consequently, the relaxed formulation $\tilde{g} : \mathbb{R}^k \rightarrow \mathbb{R}^m$ can be derived using decomposable leaf scores $\mathbf{L}_{i,:} = \prod_{j=1}^d \mathbf{l}^{(j, \beta(i, j))}$ as follows

$$\begin{aligned}
 \tilde{g}(\mathbf{x}) &= \sum_{i=1}^{b^d} \mathbf{L}_{i,:} \prod_{j=1}^d \tilde{\mathbf{S}}(\mathbf{x})_{j, \beta(i, j)} \\
 &= \sum_{i=1}^{b^d} \left(\prod_{j=1}^d \mathbf{l}^{(j, \beta(i, j))} \cdot \tilde{\mathbf{S}}(\mathbf{x})_{j, \beta(i, j)} \right) \\
 &= \sum_{a_1=1}^b \dots \sum_{a_d=1}^b \left(\prod_{j=1}^d \mathbf{l}^{(j, a_j)} \cdot \tilde{\mathbf{S}}(\mathbf{x})_{j, a_j} \right) \\
 &= \prod_{j=1}^d \left(\sum_{a_j=1}^b \mathbf{l}^{(j, a_j)} \cdot \tilde{\mathbf{S}}(\mathbf{x})_{j, a_j} \right)
 \end{aligned}$$

D. Training-Time Benchmark between GRACE and GRACE-R

The GRACE layer exhibits exponential scaling in both the number of parameters and computational cost with respect to the tree depth d and number of splits b , i.e., $\mathcal{O}(b^d)$. In contrast, the relaxed formulation GRACE-R mitigates this growth via parameter sharing in the split function and along paths to leaf nodes, yielding linear complexity $\mathcal{O}(bd)$. This reduction is demonstrated in the benchmark results below.

Setup. We benchmark training time for a fixed number of epochs while varying one structural hyperparameter at a time, either the number of splits b , the tree depth d , or the number of trees E , while keeping the others constant. For each configuration, we run the training loop three times and report the mean wall-clock time over the chosen epoch budget.

Result. Figure 6 shows the averaged training times. GRACE exhibits rapidly increasing training time as either b or d grows, consistent with the $\mathcal{O}(b^d)$ scaling. In contrast, GRACE-R maintains nearly constant training time when varying b or d , indicating that the relaxed construction effectively decouples training cost from the exponential increase in the number of leaf nodes. When increasing the number of trees E , both methods incur higher computational cost, but GRACE-R scales more modestly and remains substantially faster than GRACE for the same value of E .

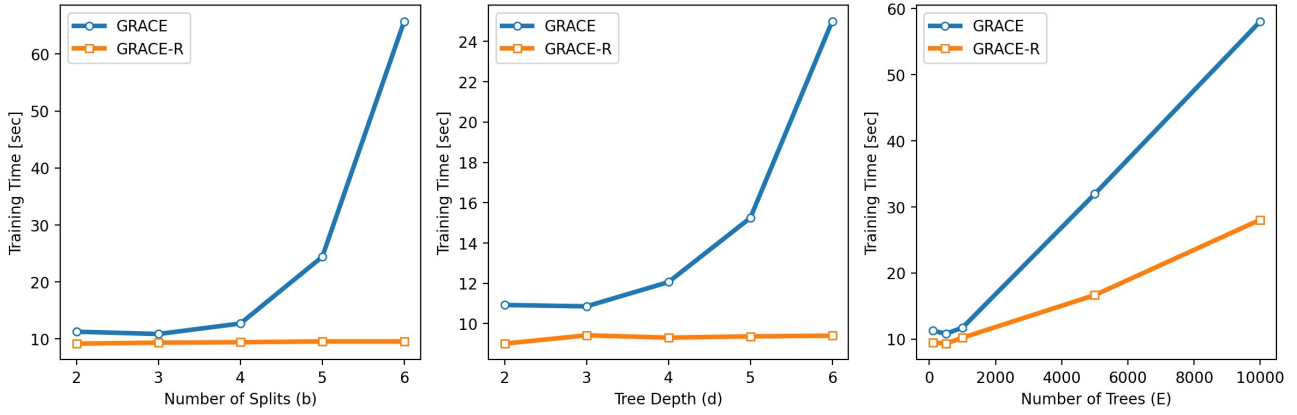


Figure 6. Performance changes resulting from varying each hyperparameter. Error bars indicate standard errors

E. Details of Datasets

E.1. Toy Dataset

A binary treatment is randomly assigned to each sample with a probability of 50%. Input features are drawn from a uniform distribution $X \sim U([0, 1]^k)$, where the feature dimension k is set to 5, 20, or 100. However, only the first two features contribute to the treatment effect defined by the function:

$$\tau(X) = \varsigma(X_1)\varsigma(X_2) \quad \text{with} \quad \varsigma(x) = 2/(1 + e^{-12(x-1/2)}) \quad (19)$$

Potential outcomes $Y(0)$ and $Y(1)$ are observed according to the treatment assignment, with additive Gaussian noise $\mathcal{N}(0, \sigma)$, where $\sigma = 0.1$ in our experiment.

E.2. Synthetic Dataset for Binary Treatment Setting

Throughout the data generating process (DGP) for the synthetic dataset, we use $k = 25$ multivariate normal features $X \sim \mathcal{N}(0, 1)^k$. X is further split into confounders X_C , which affect both outcome and treatment, prognostic factors X_P , which affect only outcome, and noise features X_N . $X_{CP} = [X_C, X_P]$ is defined as the set of all features that affect the outcome. m_C, m_P, m_{CP} denote the dimensionalities of X_C, X_P, X_{CP} , respectively.

The potential outcomes $\mu(t, x)$ are computed according to each DGP described below and then standardized. The observed outcome is then obtained by adding independent noise $\varepsilon \sim \mathcal{N}(0, 0.1)$ to the potential outcome chosen using the treatment assignment.

In terms of the treatment assignment, we first compute the quantity, $z(X)$, as given below, this is then centered and transformed into the propensity score via $\pi(X) = \sigma(z(X) - \omega)$, where $\omega = \text{median}(z(X))$. Finally, treatments are sampled as a Bernoulli random variable using the propensity score.

(i) Interaction-based DGP

To represent feature interactions up to third-order, we introduce X'_{CP} and X''_{CP} , which are randomly permuted versions of X_{CP} . We model the potential outcomes $\mu(0, x)$, $\mu(1, x)$ and $z(X)$ as

$$\begin{aligned} \mu(0, X) &= \theta_0^{(1)} \cdot X_{CP} + \theta_0^{(2)} \cdot \text{sign}(X_{CP} \circ X'_{CP}) \circ \sqrt{|X_{CP} \circ X'_{CP}|} + \theta_0^{(3)} \cdot \sqrt[3]{X_{CP} \circ X'_{CP} \circ X''_{CP}} \\ \mu(1, X) &= \theta_1^{(1)} \cdot X_{CP} + \theta_1^{(2)} \cdot \text{sign}(X_{CP} \circ X'_{CP}) \circ \sqrt{|X_{CP} \circ X'_{CP}|} + \theta_1^{(3)} \cdot \sqrt[3]{X_{CP} \circ X'_{CP} \circ X''_{CP}} \\ z(X) &= \theta_z^{(1)} \cdot X_C + \theta_z^{(2)} \cdot \text{sign}(X_C \circ X'_C) \circ \sqrt{|X_C \circ X'_C|} + \theta_z^{(3)} \cdot \sqrt[3]{X_C \circ X'_C \circ X''_C} \end{aligned}$$

where $\theta_0^{(*)}$ and $\theta_1^{(*)}$ are coefficient vectors sampled from $\mathcal{N}(0, 1)^{m_{CP}}$, and $\theta_z^{(*)}$ are coefficient vectors sampled from $\mathcal{N}(0, 1)^{m_C}$. Here, $\circ$ denotes the element-wise (Hadamard) product, and $\text{sign}(\cdot)$ is the sign function.

We set $m_C = 10$ and $m_P = 10$ in our experiment.

(ii) FCNN-based DGP

To represent a TARNet-like network structure, we further split X_P into X_{PS} , X_{P0} , and X_{P1} . With the activation function $f(\cdot) = \text{ReLU}(\cdot)$ and weight matrices $W_*^{(*)}$, where each weight is sampled from $\mathcal{N}(0, 1)$, we model the potential outcomes

$\mu(0, x)$, $\mu(1, x)$ and $z(X)$ as

$$\begin{aligned}\mu(0, X) &= f(W_0^{(3)} f(W_0^{(2)} f(W_0^{(1)} [h_C, h_{PS}, h_{P0}]))) \\ \mu(1, X) &= f(W_1^{(3)} f(W_1^{(2)} f(W_1^{(1)} [h_C, h_{PS}, h_{P1}]))) \\ z(X) &= f(W_z^{(3)} f(W_z^{(2)} f(W_z^{(1)} h_C))) \\ h_C &= f(W_C^{(3)} f(W_C^{(2)} f(W_C^{(1)} X_C))) \\ h_{PS} &= f(W_{PS}^{(3)} f(W_{PS}^{(2)} f(W_{PS}^{(1)} X_{PS}))) \\ h_{P0} &= f(W_{P0}^{(3)} f(W_{P0}^{(2)} f(W_{P0}^{(1)} X_{P0}))) \\ h_{P1} &= f(W_{P1}^{(3)} f(W_{P1}^{(2)} f(W_{P1}^{(1)} X_{P1})))\end{aligned}$$

For simplicity, and with a slight abuse of notation, we omit the bias terms. We set the hidden-layer dimension to 32, and we set $m_C, m_{PS}, m_{P0}, m_{P1}$ to 5 in our experiment.

(iii) Tree-based DGP

We construct a depth- d axis-aligned tree process to generate potential outcomes and treatment assignment. At each depth $j \in \{1, \dots, d\}$, a feature index i is selected uniformly at random from X_{CP} for the potential outcomes $\mu(0, X)$ and $\mu(1, X)$, and from X_C for the quantity $z(X)$. A threshold τ is then drawn uniformly from the empirical range of the selected feature. For each unit, Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma_{th}^2)$ is added to the threshold, and two values $(v^{(0)}, v^{(1)})$ are sampled from $\mathcal{N}(0, 1)$. At each depth, $\mu(0, X)$, $\mu(1, X)$, and $z(X)$ are updated based on whether the corresponding feature exceeds the perturbed threshold.

Formally, for each depth j , we apply the following updates and repeat the process until depth d :

$$\begin{aligned}\mu(0, X) &\leftarrow \mu(0, X) + v_0^{(0)} \cdot \mathbb{1}\{X_{CP, i_0} > \tau_0 + \epsilon_0\} + v_0^{(1)} \cdot \mathbb{1}\{X_{CP, i_0} \leq \tau_0 + \epsilon_0\} \\ \mu(1, X) &\leftarrow \mu(1, X) + v_1^{(0)} \cdot \mathbb{1}\{X_{CP, i_1} > \tau_1 + \epsilon_1\} + v_1^{(1)} \cdot \mathbb{1}\{X_{CP, i_1} \leq \tau_1 + \epsilon_1\} \\ z(X) &\leftarrow z(X) + v_z^{(0)} \cdot \mathbb{1}\{X_{C, i_z} > \tau_z + \epsilon_z\} + v_z^{(1)} \cdot \mathbb{1}\{X_{C, i_z} \leq \tau_z + \epsilon_z\}\end{aligned}$$

We set the tree depth to $d = 10$, and use a threshold-noise variance of $\sigma_{th}^2 = 0.1$ in our experiment. We further set $m_C = 10$ and $m_P = 10$.

E.3. Synthetic Dataset for Non-binary Treatment Setting

Similar to the binary treatment setting, we use $k = 25$ multivariate normal features $X \sim \mathcal{N}(0, 1)^k$, which are split into confounders X_C , prognostic factors X_P , and noise features X_N .

For treatment assignment, we first compute an m_t -dimensional vector $z(X)$ using a process similar to that in the (i) Interaction-based DGP:

$$z(X) = \Theta^{(1)} \cdot X_C + \Theta^{(2)} \cdot \text{sign}(X_C \circ X'_C) \circ \sqrt{|X_C \circ X'_C|} + \Theta^{(3)} \cdot \sqrt[3]{X_C \circ X'_C \circ X''_C}$$

where $\Theta^{(1)}$, $\Theta^{(2)}$, and $\Theta^{(3)}$ are coefficient matrices sampled from $\mathcal{N}(0, 1)^{m_t \times m_C}$.

After $z(X)$ is standardized, it is then transformed and treatment vector $\mathbf{t}$ is sampled depending on the treatment setting. For **(iv) Multi-valued Treatment** setting, $z(X)$ is transformed into category probability via a softmax function, and treatments are sampled from a Categorical distribution. The resulting treatment indices are then converted into a one-hot vector format. For **(v) Continuous Treatment** setting, $z(X)$ is directly used as the continuous treatment vector. For **(vi) Bundle Treatment** setting, $z(X)$ is transformed into occurrence probability using a sigmoid function, and each treatment component is sampled independently from a Bernoulli distribution.

To account for interactions between the treatment and the feature, as well as interactions among treatment themselves, the treatment vector $\mathbf{t}$ is concatenated with the feature vector X_{CP} to construct $X_{CPT} = [X_{CP}, \mathbf{t}]$. In the similar manner to the (i) Interaction-base DGP, the potential outcome is computed as:

$$\mu(\mathbf{t}, X) = \theta^{(1)} \cdot X_{CPT} + \theta^{(2)} \cdot \text{sign}(X_{CPT} \circ X'_{CPT}) \circ \sqrt{|X_{CPT} \circ X'_{CPT}|} + \theta^{(3)} \cdot \sqrt[3]{X_{CPT} \circ X'_{CPT} \circ X''_{CPT}} \quad (20)$$

where $\theta^{(0)}$, $\theta^{(1)}$, and $\theta^{(2)}$ are coefficient vectors sampled from $\mathcal{N}(0, 1)^{m_{CPT}}$, X'_{CPT} and X''_{CPT} are randomly permuted versions of X_{CPT} . The observed outcome obtained by adding independent noise $\varepsilon \sim \mathcal{N}(0, 0.1)$ to each sample.

We set $m_C = 10$ and $m_P = 10$ in our experiment.

E.4. Semi-synthetic Dataset

IHDP (The Infant Health and Development Program)

The IHDP dataset is one of the most popular dataset for causal inference. It was first introduced in Hill (2011), and it has been used as a benchmark in a lot of technical work (Yoon et al., 2018; Hassanpour & Greiner, 2019; Curth & Schaar, 2021; Chu et al., 2021) since then. The original data comes from a randomized controlled study (Brooks-Gunn et al., 1992) designed to evaluate the effect of home visit from specialist doctors on the cognitive test scores of premature infants, with the original outcome of cognitive test scores being replaced by the data generating process as described below.

To simulate the observational setting, confounding bias is introduced by simply throwing away all samples with non-white mothers only from the treatment group. Out of 103 variables measured in the original study, only 25 variables are selected as covariates of interest. The function to generate outcomes, also known as response surface, is defined as the following expressions:

$$\begin{aligned} Y(0) &= \exp((X + 0.5)\beta + \epsilon_0) & \epsilon_0 &\sim N(0, 1) \\ Y(1) &= X\beta - \omega + \epsilon_1 & \epsilon_1 &\sim N(0, 1) \end{aligned}$$

where X is a matrix comprising the standardized covariates for all samples, $\beta \in \mathbb{R}^{25}$ is a vector of coefficients randomly sampled from values (0.0, 0.1, 0.2, 0.3, 0.4) with probabilities (0.6, 0.1, 0.1, 0.1, 0.1) respectively, $\omega \in \mathbb{R}$ is chosen such that CATT (Conditional Average Treatment effect on the Treated) equals 4.0.

To prevent performance from being dominated by runs with high CATE variance, we apply the rescaling procedure proposed in Curth & Schaar (2021). When the variance of the CATE in the training data satisfies $\sigma_{CATE}^2 > 1$, we rescale the expected potential outcomes as $\tilde{\mu}(t, x) = \mu(t, x) / \sigma_{CATE}$. Furthermore, to enable the calculation of AUC and Regret, we add a bias to $\mu(1, x)$ such that the median of the CATE distribution is equal to 0.

The original data of IHDP and the original code of preprocessing can be found in R npci package².

ACIC 2016 (Atlantic Causal Inference Conference Data Analysis Challenge 2016)

ACIC 2016 dataset is used in 2016 Data Analysis Challenge associated with the Atlantic Causal Inference Conference (ACIC)³. The purpose of the challenge is to perform a blinded empirical evaluation of causal inference methods on synthetic data, where the true data generating process is revealed after estimation and inference accuracy for each method has been determined.

Treatment assignment and response surface are simulated from real-world data corresponding to 4802 individuals and 58 features. Unlike IHDP dataset, functional forms for treatment assignment and response surface are variegated depending on the settings. Out of 77 settings used in the actual challenge, we includ 75 in our evaluation. The remaining 2 settings exhibit no heterogeneity in the treatment effect, and therefore neither AUC nor Regret can be computed for them.

²<https://github.com/vdorie/npqi>

³<https://jenniferhill17.wixsite.com/acic-2016/competition>

We also apply the same rescaling and bias-adjustment procedure used for the IHDP dataset to the ACIC 2016 dataset.

The detailed description for the data generating process can be found in [Dorie et al. \(2019\)](#). The original data of ACIC 2016 and the original code of preprocessing can be found in a GitHub repository⁴.

TCGA (The Cancer Genome Atlas Program) and News

We follow the data generating process proposed in SCIGAN ([Bica et al., 2020](#)).

We obtain features, $\mathbf{x}$, from the real datasets TCGA and News provided by DRNet ([Schwab et al., 2020](#)). We consider 3 treatments each accompanied by a dosage. Each treatment, w , is associated with a set of parameters, $\mathbf{v}_1^w, \mathbf{v}_2^w, \mathbf{v}_3^w$. For each run of the experiment, these parameters are sampled randomly by sampling a vector, $\mathbf{u}_i^w$, from $\mathcal{U}([0, 10])$ and then setting $\mathbf{v}_i^w = \mathbf{u}_i^w / \|\mathbf{u}_i^w\|$ where $\|\cdot\|$ is Euclidean norm. The shape of the response curve for each treatment, $f_w(\mathbf{x}, d)$, is given in Table 3, along with a closed-form expression for the optimal dosage. We add $\mathcal{N}(0, 0.5)$ noise to the outcomes.

We assign interventions by sampling a dosage, d_w , for each treatment from a beta distribution, $d_w | \mathbf{x} \sim \text{Beta}(\alpha, \beta_w)$. $\alpha \geq 1$ controls the dosage selection bias ($\alpha = 1$ gives the uniform distribution). $\beta_w = \frac{\alpha-1}{d_w^*} + 2 - \alpha$, where d_w^* is the optimal dosage for treatment w . This setting of β_w ensures that the mode of $\text{Beta}(\alpha, \beta_w)$ is d_w^* . We then assign a treatment according to $w_f | \mathbf{x} \sim \text{Categorical}(\text{softmax}(\kappa f(\mathbf{x}, d_w)))$ where increasing κ increases selection bias, and $\kappa = 0$ leads to random assignments. The factual intervention is given by (w_f, d_{w_f}) . We set $\kappa = 2$ and $\alpha = 2$.

Table 3. Dose response curves used to generate semi-synthetic outcomes for patient features $\mathbf{x}$. In the experiments, we set $C = 10$. $\mathbf{v}_1^w, \mathbf{v}_2^w, \mathbf{v}_3^w$ are the parameters associated with each treatment w .

Treatment	Dose-Response	Optimal dosage
1	$f_1(\mathbf{x}, d) = C((\mathbf{v}_1^1)^T \mathbf{x} + 12(\mathbf{v}_2^1)^T \mathbf{x}d - 12(\mathbf{v}_3^1)^T \mathbf{x}d^2)$	$d_1^* = \frac{(\mathbf{v}_2^1)^T \mathbf{x}}{2(\mathbf{v}_3^1)^T \mathbf{x}}$
2	$f_2(\mathbf{x}, d) = C((\mathbf{v}_1^2)^T \mathbf{x} + \sin(\pi(\frac{(\mathbf{v}_2^2)^T \mathbf{x}}{(\mathbf{v}_3^2)^T \mathbf{x}}d))$	$d_2^* = \frac{(\mathbf{v}_3^2)^T \mathbf{x}}{2(\mathbf{v}_2^2)^T \mathbf{x}}$
3	$f_3(\mathbf{x}, d) = C((\mathbf{v}_1^3)^T \mathbf{x} + 12d(d - b)^2, \text{ where } b = 0.75 \frac{(\mathbf{v}_3^3)^T \mathbf{x}}{(\mathbf{v}_2^3)^T \mathbf{x}})$	$\begin{cases} \frac{b}{3} & \text{if } b \geq 0.75 \\ 1 & \text{if } b < 0.75 \end{cases}$

Glucose

We use *simglucose*⁵, a Python implementation of the FDA-approved glucose simulator. It provides a pharmacokinetic model along with parameter sets for 30 virtual patients. Although *simglucose* was originally developed for reinforcement learning applications, we use it as a one-step decision-making environment for HTE estimation. Specifically, we assume that patient-specific features are first obtained, a treatment is then selected, and the resulting glucose level is measured one hour later.

The feature vector comprised 17 variables, including both patient characteristics and initial system states. The treatment space consists of combinations of three components: basal insulin infusion, bolus insulin dosing, and meal intake. The outcome is defined as the glucose value measured one hour after the initial observation. To generate more than 30 patient scenarios, we randomly sample the model parameters of one of the 30 virtual patients, while overwriting the initial system states with samples drawn from age-dependent distributions reported in [Esponera & Cinà \(2025\)](#). Glucose values used for model training are obtained from a sensor that includes measurement noise, whereas evaluation metrics are computed using the true (noise-free) glucose values. All features are standardized as a preprocessing step.

For binary-treatment experiments, we generate treatment assignments using a propensity score transformed from the standardized patient-specific features $\mathbf{X}$, defined as:

$$\pi(X) = \sigma\left(\frac{1}{\gamma} w^T X\right) \quad (21)$$

where w is a coefficient vector drawn from a Gaussian distribution $\mathcal{N}(0, 1)$, and γ is a parameter controlling degree of the selection bias. We set $\gamma = 3$ in this experiment. Treatment is then sampled from a Bernoulli distribution with success

⁴<https://github.com/vdorie/aciccomp>

⁵<https://github.com/jxx123/simglucose>

probability $\pi(X)$. In this setup, treatment 0 corresponded to no basal insulin infusion, whereas treatment 1 corresponded to administering a basal dose of 1.0 unit.

For the non-binary treatment setting, the treatment vector consisted of four continuous variables: meal timing, meal volume, basal insulin rate, and bolus insulin dose. The bolus dose is administered at the same time as the meal. Each treatment variable is generated by independently sampling from a Beta distribution whose parameters are derived from transformations of the standardized patient-specific features X :

$$\begin{aligned} t_{\text{time}}|X &\sim \alpha_{\text{time}} \cdot \text{Beta}(1 + 3z(X), 4 - 3z(X)) \\ t_{\text{meal}}|X &\sim \alpha_{\text{meal}} \cdot \text{Beta}(1 + 3z(X), 4 - 3z(X)) \\ t_{\text{basal}}|X &\sim \alpha_{\text{basal}} \cdot \text{Beta}(1 + 3z(X), 4 - 3z(X)) \\ t_{\text{bolus}}|X &\sim \alpha_{\text{bolus}} \cdot \text{Beta}(1 + 3z(X), 4 - 3z(X)) \end{aligned} \quad (22)$$

$$\text{with } z(X) = \sigma \left(\frac{1}{\gamma} w^T X \right)$$

where w is a coefficient vector drawn from a Gaussian distribution $\mathcal{N}(0, 1)$, and γ is a parameter controlling degree of the selection bias. We set $[\alpha_{\text{time}}, \alpha_{\text{meal}}, \alpha_{\text{basal}}, \alpha_{\text{bolus}}] = [60, 100, 0.1, 5.0]$ and $\gamma = 10$ in this experiment. All sampled treatment values are subsequently standardized as a preprocessing step.

E.5. Summary

The datasets used for evaluation are summarized in Table 4.

Table 4. Summary of Datasets

Dataset	Treatment	Training Sample Size	Treatment Dimension	Feature Dimension
Toy	Binary	5,000	1	[5, 20, 100]
(i) Interaction-based DGP	Binary	[500, 1,000, 5,000, 10,000]	1	25
(ii) FCNN-based DGP	Binary	[500, 1,000, 5,000, 10,000]	1	25
(iii) Tree-based DGP	Binary	[500, 1,000, 5,000, 10,000]	1	25
(iv) Multi-valued Treatment	Multi-valued	[500, 1,000, 5,000, 10,000]	5	25
(v) Continuous Treatment	Continuous	[500, 1,000, 5,000, 10,000]	1	25
(vi) Bundle Treatment	Bundle	[500, 1,000, 5,000, 10,000]	5	25
IHDP	Binary	[100, 200, 500, 627]	1	25
ACIC 2016	Binary	[100, 200, 500, 1,000, 4321]	1	58
Glucose (Binary)	Binary	[100, 200, 500, 1,000]	1	60
TCGA	Multi-dimensional	[500, 1,000, 5,000, 8,693]	2	4000
News	Multi-dimensional	[500, 1,000, 5,000, 9,000]	2	2858
Glucose (Non-binary)	Multi-dimensional	[100, 200, 500, 1,000]	4	17

F. Details of Baselines

In this section, we describe the implementation of each baseline.

- **Bayesian Additive Regression Trees (BART)** (Chipman et al., 2010; Hill, 2011). We use Python StochTree package⁶. The treatment variable is concatenated with the feature vector and used as input to the model. In the binary treatment setting, the propensity score estimated by BART is also concatenated with the input features. We perform PCA to reduce the dimensionality to 50 to ensure computational tractability when the dimensionality of the feature vector exceeds 100 (applied only to the TCGA and News datasets).
- **TransTEE** (Zhang et al., 2023). We use the Python implementation provided by the authors⁷. We follow the authors' implementation, in which a fully-connected layer is added before the embedding layer to ensure computational tractability when the dimensionality of the feature vector exceeds 100 (applied only to the TCGA and News datasets).
- **Causal Forest (CF)** (Wager & Athey, 2018). We use Python EconML package⁸ and implement Causal Forest using econml.grf.CausalForest class.
- **Causal Bayesian Forest (BCF)** (Hahn et al., 2020). We use Python StochTree package⁶. The propensity score is included as a covariate in both treatment effect model and prognostic model.
- **T-Learner (Künzel et al., 2019) with Random Forest (T-RF)**. We use Python EconML package⁸ and implement Random Forest using econml.grf.RegressionForest class.
- **TARNet (Shalit et al., 2017)**. We implement TARNet with PyTorch.
- **CFRNet (Shalit et al., 2017)**. We implement CFRNet with PyTorch.
- **DragonNet (Shi et al., 2019)**. We implement DragonNet using PyTorch. Following the original paper, we include both the loss term for propensity score head and the targeted regularization.
- **SNet (Curth & Schaar, 2021)**. We implement SNet using PyTorch. Following the original paper, we include both the loss term for propensity score head and the orthogonal loss.
- **DIGNet (Huang et al., 2024)**. We use the Python implementation provided by the authors of DIGNet⁹.
- **ESCFR (Wang et al., 2023)**. We use the Python implementation provided by the authors of ESCFR¹⁰.
- **S-Learner (Künzel et al., 2019) with Random Forest (S-RF)**. We use Python EconML package⁸ and implement Random Forest using econml.grf.RegressionForest class. The treatment variable is concatenated with the feature vector and used as input to the model. We perform PCA to reduce the dimensionality to 50 to ensure computational tractability when the dimensionality of the feature vector exceeds 100 (applied only to the TCGA and News datasets).
- **S-Learner (Künzel et al., 2019) with Neural Network (S-NN)**. We implement S-Learner with Neural Network using PyTorch. The treatment variable is concatenated with the feature vector and used as input to the model.
- **NCoRE (Parbhoo et al., 2021)**. We implement NCoRE using PyTorch.
- **RMNet (Tanimoto et al., 2021)**. We implement RMNet using PyTorch. Following the original paper, we include both the ranking term and the balancing regularizer.
- **VSR (Zou et al., 2020)**. We use the Python implementation provided by the authors of SCP (Qian et al., 2021)¹¹.
- **SCP (Qian et al., 2021)**. We use the Python implementation provided by the authors¹¹.
- **VCNet (Nie et al., 2020)**. We use the Python implementation provided by the authors¹².

⁶<https://stochtree.ai/index.html>

⁷<https://github.com/hlzhang109/TransTEE>

⁸<https://www.pywhy.org/EconML/>

⁹<https://openreview.net/forum?id=Z20FInfWlm>

¹⁰<https://openreview.net/forum?id=DAdfU1ASLb>

¹¹<https://github.com/ZhaozhiQIAN/Single-Cause-Perturbation-NeurIPS-2021>

¹²https://github.com/lushleaf/varying-coefficient-net-with-functional-tr/blob/main/models/dynamic_net.py

The baselines we used in our experiments can be summarized below.

Table 5. Baselines

	Toy	(i) Interaction-based DGP	(ii) FCNN-based DGP	(iii) Tree-based DGP	(iv) Multi-valued Treatment	(v) Continuous Treatment	(vi) Bundle Treatment	IHDP	ACIC 2016	TCGA	News	Glucose (Binary)	Glucose (Non-binary)
BART (Chipman et al., 2010; Hill, 2011)		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
TransTEE (Zhang et al., 2023)		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
CF (Wager & Athey, 2018)	✓	✓	✓	✓				✓	✓			✓	
BCF (Hahn et al., 2020)		✓	✓	✓				✓	✓			✓	
T-RF (Künzel et al., 2019)		✓	✓	✓				✓	✓			✓	
TARNet (Shalit et al., 2017)	✓	✓	✓	✓				✓	✓			✓	
CFRNet (Shalit et al., 2017)		✓	✓	✓				✓	✓			✓	
DragonNet (Shi et al., 2019)		✓	✓	✓				✓	✓			✓	
SNet (Curth & Schaar, 2021)		✓	✓	✓				✓	✓			✓	
DIGNet (Huang et al., 2024)		✓	✓	✓				✓	✓			✓	
ESCFR (Wang et al., 2023)		✓	✓	✓				✓	✓			✓	
S-RF (Künzel et al., 2019)					✓	✓	✓			✓	✓		✓
S-NN (Künzel et al., 2019)					✓	✓	✓			✓	✓		✓
NCoRE (Parbhoo et al., 2021)					✓		✓						
RMNet (Tanimoto et al., 2021)					✓		✓						
VSR (Zou et al., 2020)					✓		✓						
SCP (Qian et al., 2021)					✓		✓						
VCNet (Nie et al., 2020)						✓							

G. Hyper-parameter Tuning

We tune the hyper-parameters using Optuna’s TPE sampler (Akiba et al., 2019). We select the best hyper-parameter configuration based on the averaged factual loss on a validation dataset over 3 randomly sampled simulation runs, after conducting 50 tuning trials. Hyper-parameter tuning is performed once for each dataset, and resulting configuration is reused for all training sample size on the same dataset. The training sample sizes used during tuning are 5,000 for the toy dataset, synthetic datasets, TCGA, News dataset, and 500 for IHDP, ACIC 2016, and Glucose datasets.

We tune the following hyper-parameters using the range below:

Table 6. Hyper-parameters

Method	Hyper-parameter	Range
BART BCF	Number of trees	[100, 500, 1,000]
	Maximum tree depth	[4, 5, 6, 7, 8, 9, 10]
	Number of GFR iterations	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
	Number of MCMC iterations	[10, 50, 100, 500]
T-RF S-RF CF	Number of trees	[100, 500, 1,000]
	Maximum tree depth	[4, 5, 6, 7, 8, 9, 10]
	Minimum samples for an internal node	[5, 10, 15]
	Minimum samples for a leaf node	[5, 10, 15]
	Subsample ratio	[0.3, 0.35, 0.4, 0.45, 0.5]
	Honest tree	[True, False]
	Split criterion (CF only)	[MSE, Heterogeneity Score]
TARNet CFRNet DragonNet SNet DIGNet ESCFR NCoRE RMNet	Number of hidden units for shared layer	[16, 32, 64, 128]
	Number of hidden units for dedicated layers	[16, 32, 64, 128]
	Number of layers	[1, 2, 3]
	Weight decay of Adam	[0.00001, 0.0001, 0.001, 0.01, 0.1]
	Dropout rate	[0.0, 0.1, 0.2, 0.3]
	Non-linear activation function	[ELU, ReLU, Leaky ReLU, SELU]
	Weighting coefficient for imbalance loss (CFRNet and RMNet only)	[0.001, 0.01, 0.1, 1.0]
	Weighting coefficient for propensity score loss (DragonNet and SNet only)	[0.001, 0.01, 0.1, 1.0]
	Weighting coefficient for targeted regularization (DragonNet only)	[0.001, 0.01, 0.1, 1.0]
	Weighting coefficient for orthogonal loss (SNet only)	[0.001, 0.01, 0.1, 1.0]
S-NN VSR SCP VCNet	Weighting coefficient for GNet / INet (DIGNet only)	[0.001, 0.01, 0.1, 1.0]
	Hyperparameters of ESCFR ($\lambda, \epsilon, \kappa, \lambda$) (ESCFR only)	[0.001, 0.01, 0.1, 1.0]
	Number of hidden units	[16, 32, 64, 128]
	Number of layers	[2, 3, 4, 5, 6]
S-NN VSR SCP VCNet	Weight decay of Adam	[0.00001, 0.0001, 0.001, 0.01, 0.1]
	Dropout rate	[0.0, 0.1, 0.2, 0.3]
	Non-linear activation function	[ELU, ReLU, Leaky ReLU, SELU]
	Number of knots (VCNet only)	[5, 10, 15, 20]
	Weighting coefficient for conditional density loss (VCNet only)	[0.5, 1.0]
TransTEE	Learning rate	[0.0001, 0.0005, 0.001, 0.005, 0.01]
	Weight decay of Adam	[0.00001, 0.0001, 0.001, 0.01, 0.1]
	Embedding size	[16, 32, 64, 128]
	Number of hidden units	[16, 32, 64, 128]
	Number of attention layers	[1, 2, 3, 4]
	Dropout rate	[0.0, 0.1, 0.2, 0.3]
GRACE GRACE-R	Number of trees (E)	[100, 500, 1,000]
	Tree depth (d)	[3, 4]
	Number of splits (b)	[2, 3, 4]
	Learning rate	[0.005, 0.01, 0.05]
	Feature sampling ratio (λ)	[0.5, 0.6, 0.7, 0.8, 0.9, 1.0]
	Dropout rate (ρ)	[0.0, 0.1, 0.2, 0.3, 0.4, 0.5]
	Number of hidden units (h)	[16, 32, 64, 128]

H. Hyper-parameter Sensitivity Analysis

In HTE estimation, the true individual-level treatment effects are fundamentally unobservable, making it difficult to evaluate model specification or tuning decisions. As a result, algorithms that require finely tuned hyper-parameters are generally undesirable. Consequently, demonstrating robustness to hyper-parameter choices is essential for ensuring that a method can be reliably applied in practical HTE estimation tasks.

Setup. We conduct a comprehensive hyperparameter sensitivity analysis for both GRACE and GRACE-R across the four datasets used in the binary treatment setting (three synthetic datasets and one semi-synthetic dataset). Starting from the best-performing hyper-parameter configuration identified through our hyperparameter search, we vary each hyperparameter individually while keeping all others fixed. For each perturbed hyperparameter setting, we run three independent simulation runs and report the mean and standard error of the performance metrics. This procedure enables us to isolate the marginal effect of each hyperparameter on model performance and assess the robustness of the methods.

Result. Figure 7 shows the performance changes induced by varying each hyper-parameter across the different datasets. Overall, both GRACE and GRACE-R exhibit low sensitivity to hyperparameter choices across all four datasets. Although increasing the Number of Trees (E), Tree Depth (d), and Number of Splits (b) tends to yield slightly improved performance, these patterns are not strongly pronounced. For the remaining hyper-parameters, including the Feature Sampling Ratio (λ), Dropout Rate (ρ), and Number of Hidden Units (h), we do not observe consistent performance trends. These results demonstrate that both models are robust to moderate hyper-parameter variations, suggesting that practitioners can rely on reasonable default values without extensive hyperparameter tuning. Moreover, the observed robustness may reflect structural properties of the GRACE architecture. Because both GRACE and GRACE-R aggregate information across multiple trees and incorporate representation-learning components that naturally smooth over local fluctuations, moderate perturbations in hyper-parameters may have limited impact on the final estimates. This stability is particularly desirable in HTE estimation, where model misspecification cannot be directly diagnosed due to the unobservability of the true treatment effects.

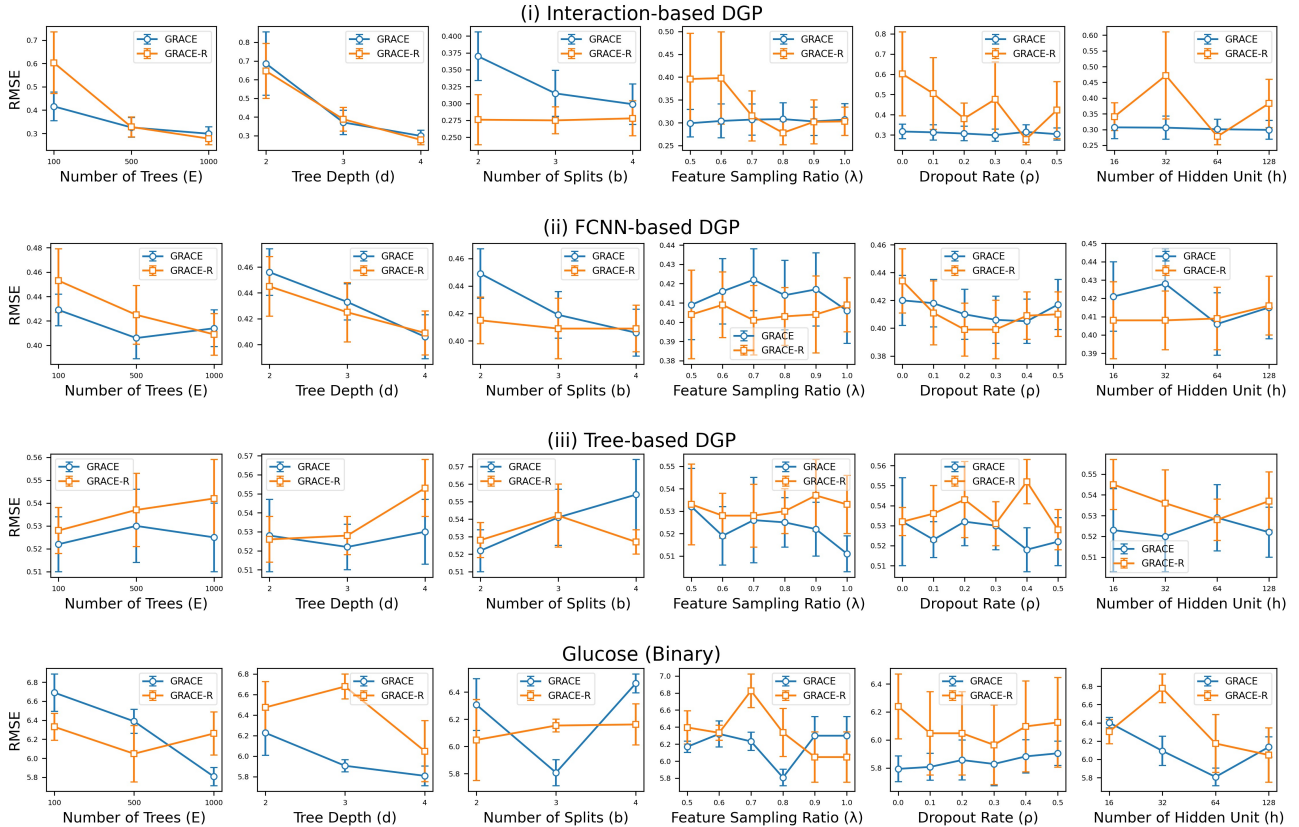


Figure 7. Performance changes resulting from varying each hyperparameter. Error bars indicate standard errors

I. Full Results

I.1. (i) Interaction-based DGP

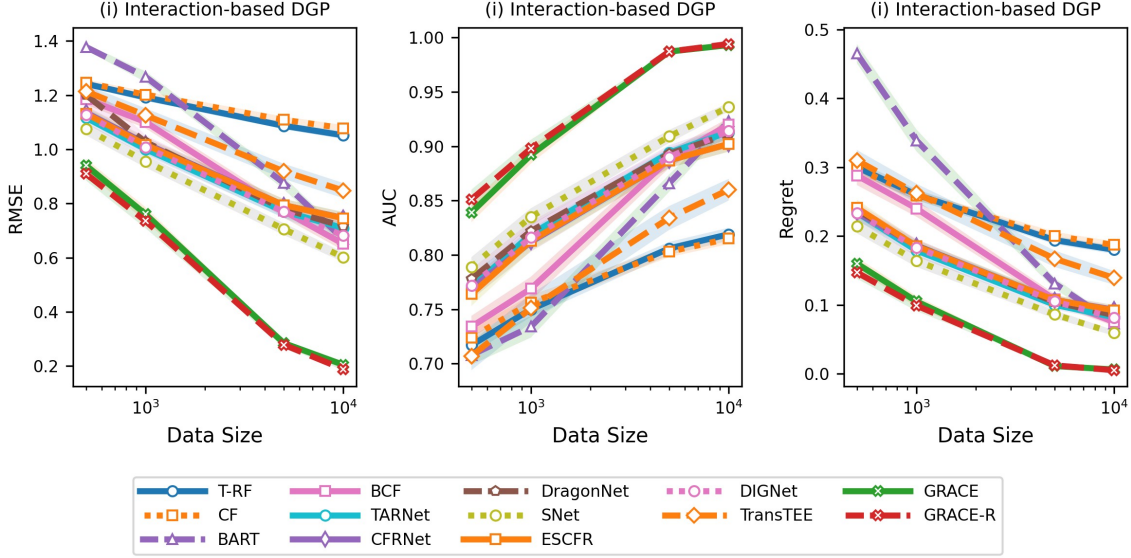


Figure 8. Performance comparison for (i) **Interaction-based DGP**. The shaded areas indicate standard error.

Figure 9. Full result for (i) **Interaction-based DGP**. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	10000
RMSE	T-RF	1.242 (0.012)	1.192 (0.013)	1.087 (0.013)	1.052 (0.014)
	CF	1.246 (0.012)	1.202 (0.012)	1.109 (0.012)	1.078 (0.012)
	BART	1.378 (0.011)	1.268 (0.020)	0.876 (0.023)	0.669 (0.020)
	BCF	1.186 (0.021)	1.101 (0.023)	0.773 (0.025)	0.651 (0.024)
	TARNet	1.117 (0.023)	1.001 (0.023)	0.771 (0.023)	0.710 (0.020)
	CFRNet	1.136 (0.024)	1.022 (0.024)	0.794 (0.023)	0.747 (0.022)
	DragonNet	1.201 (0.054)	1.027 (0.026)	0.780 (0.022)	0.716 (0.019)
	SNet	1.076 (0.024)	0.956 (0.023)	0.706 (0.022)	0.602 (0.018)
	ESCFR	1.132 (0.024)	1.017 (0.023)	0.794 (0.023)	0.746 (0.022)
	DIGNet	1.128 (0.025)	1.008 (0.024)	0.770 (0.024)	0.683 (0.021)
	TransTEE	1.214 (0.024)	1.126 (0.026)	0.919 (0.028)	0.848 (0.028)
	GRACE	0.941 (0.024)	0.761 (0.026)	0.285 (0.007)	0.205 (0.004)
	GRACE-R	0.910 (0.025)	0.736 (0.027)	0.279 (0.008)	0.189 (0.004)
AUC	T-RF	0.717 (0.008)	0.750 (0.007)	0.806 (0.006)	0.819 (0.006)
	CF	0.724 (0.008)	0.756 (0.007)	0.803 (0.005)	0.815 (0.005)
	BART	0.708 (0.009)	0.734 (0.010)	0.866 (0.008)	0.923 (0.005)
	BCF	0.734 (0.010)	0.769 (0.011)	0.888 (0.008)	0.920 (0.006)
	TARNet	0.770 (0.010)	0.819 (0.009)	0.894 (0.007)	0.913 (0.006)
	CFRNet	0.769 (0.010)	0.812 (0.009)	0.887 (0.007)	0.902 (0.007)
	DragonNet	0.778 (0.009)	0.822 (0.009)	0.893 (0.006)	0.911 (0.005)
	SNet	0.789 (0.009)	0.835 (0.008)	0.909 (0.006)	0.936 (0.004)
	ESCFR	0.764 (0.011)	0.813 (0.009)	0.887 (0.007)	0.902 (0.007)
	DIGNet	0.772 (0.010)	0.816 (0.008)	0.890 (0.008)	0.914 (0.006)
	TransTEE	0.707 (0.013)	0.751 (0.013)	0.834 (0.011)	0.860 (0.010)
	GRACE	0.839 (0.009)	0.892 (0.008)	0.987 (0.001)	0.993 (0.000)
	GRACE-R	0.851 (0.009)	0.898 (0.008)	0.987 (0.001)	0.994 (0.000)
Regret	T-RF	0.300 (0.009)	0.260 (0.008)	0.194 (0.006)	0.180 (0.006)
	CF	0.298 (0.008)	0.259 (0.007)	0.200 (0.006)	0.187 (0.005)
	BART	0.465 (0.014)	0.339 (0.016)	0.131 (0.008)	0.073 (0.005)
	BCF	0.288 (0.014)	0.240 (0.012)	0.108 (0.008)	0.075 (0.006)
	TARNet	0.234 (0.011)	0.179 (0.009)	0.100 (0.007)	0.082 (0.006)
	CFRNet	0.234 (0.011)	0.186 (0.009)	0.107 (0.007)	0.093 (0.007)
	DragonNet	0.238 (0.011)	0.186 (0.010)	0.104 (0.006)	0.085 (0.005)
	SNet	0.214 (0.010)	0.164 (0.009)	0.086 (0.006)	0.059 (0.004)
	ESCFR	0.241 (0.012)	0.185 (0.009)	0.107 (0.007)	0.092 (0.007)
	DIGNet	0.233 (0.010)	0.183 (0.009)	0.105 (0.008)	0.081 (0.006)
	TransTEE	0.310 (0.015)	0.262 (0.014)	0.167 (0.012)	0.139 (0.010)
	GRACE	0.160 (0.009)	0.105 (0.008)	0.011 (0.001)	0.006 (0.000)
	GRACE-R	0.147 (0.009)	0.099 (0.008)	0.012 (0.001)	0.005 (0.000)

I.2. (ii) FCNN-based DGP

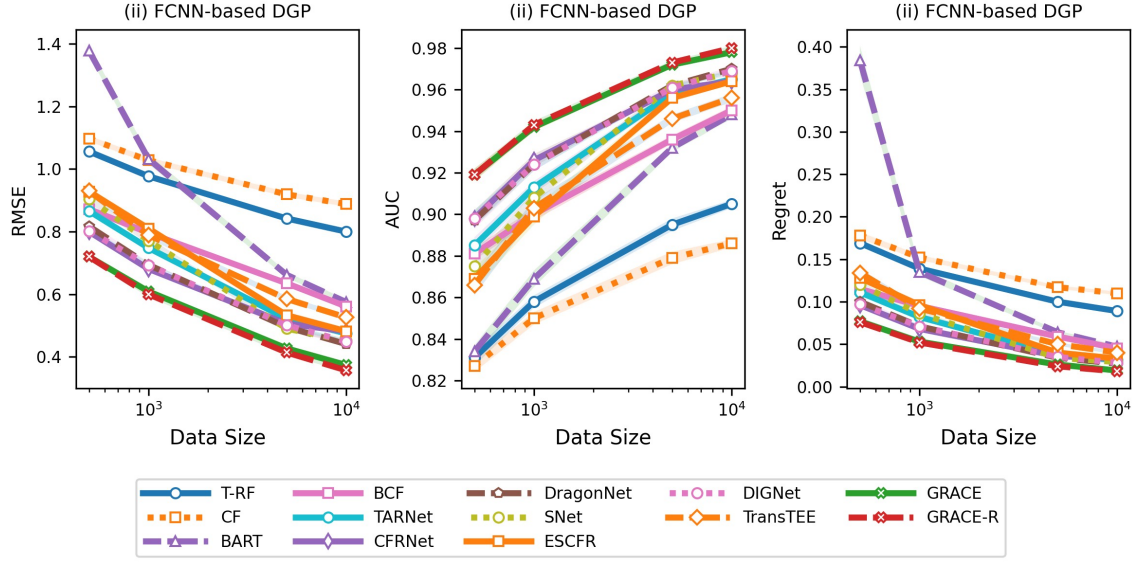


Figure 10. Performance comparison for (ii) FCNN-based DGP. The shaded areas indicate standard error.

Figure 11. Full result for (ii) FCNN-based DGP. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	10000
RMSE	T-RF	1.056 (0.009)	0.977 (0.009)	0.842 (0.008)	0.800 (0.008)
	CF	1.097 (0.009)	1.028 (0.009)	0.919 (0.008)	0.888 (0.008)
	BART	1.378 (0.015)	1.032 (0.016)	0.662 (0.009)	0.575 (0.008)
	BCF	0.872 (0.010)	0.796 (0.010)	0.635 (0.008)	0.561 (0.008)
	TARNet	0.866 (0.012)	0.748 (0.010)	0.522 (0.008)	0.475 (0.007)
	CFRNet	0.800 (0.011)	0.681 (0.009)	0.506 (0.007)	0.482 (0.006)
	DragonNet	0.817 (0.012)	0.694 (0.009)	0.495 (0.007)	0.441 (0.006)
	SNet	0.904 (0.012)	0.771 (0.011)	0.490 (0.008)	0.452 (0.007)
	ESCFR	0.928 (0.013)	0.810 (0.012)	0.533 (0.008)	0.482 (0.007)
	DIGNet	0.802 (0.010)	0.693 (0.009)	0.501 (0.007)	0.449 (0.006)
	TransTEE	0.930 (0.014)	0.789 (0.012)	0.585 (0.009)	0.526 (0.008)
	GRACE	0.720 (0.010)	0.609 (0.009)	0.427 (0.007)	0.374 (0.006)
	GRACE-R	0.720 (0.010)	0.600 (0.009)	0.414 (0.007)	0.357 (0.006)
AUC	T-RF	0.831 (0.004)	0.858 (0.003)	0.895 (0.003)	0.905 (0.002)
	CF	0.827 (0.003)	0.850 (0.003)	0.879 (0.003)	0.886 (0.002)
	BART	0.834 (0.003)	0.869 (0.004)	0.932 (0.002)	0.948 (0.002)
	BCF	0.881 (0.003)	0.900 (0.003)	0.936 (0.002)	0.950 (0.002)
	TARNet	0.885 (0.004)	0.913 (0.003)	0.958 (0.002)	0.965 (0.001)
	CFRNet	0.898 (0.004)	0.926 (0.003)	0.960 (0.001)	0.964 (0.001)
	DragonNet	0.897 (0.004)	0.924 (0.003)	0.962 (0.001)	0.970 (0.001)
	SNet	0.875 (0.004)	0.908 (0.003)	0.962 (0.001)	0.968 (0.001)
	ESCFR	0.869 (0.005)	0.899 (0.004)	0.956 (0.002)	0.964 (0.001)
	DIGNet	0.898 (0.004)	0.924 (0.003)	0.961 (0.001)	0.969 (0.001)
	TransTEE	0.866 (0.005)	0.903 (0.004)	0.946 (0.002)	0.956 (0.002)
	GRACE	0.919 (0.003)	0.942 (0.002)	0.972 (0.001)	0.978 (0.001)
	GRACE-R	0.919 (0.003)	0.943 (0.002)	0.973 (0.001)	0.980 (0.001)
Regret	T-RF	0.169 (0.003)	0.139 (0.003)	0.100 (0.002)	0.089 (0.002)
	CF	0.178 (0.004)	0.152 (0.003)	0.117 (0.002)	0.110 (0.002)
	BART	0.384 (0.017)	0.135 (0.005)	0.064 (0.002)	0.047 (0.002)
	BCF	0.115 (0.003)	0.095 (0.003)	0.059 (0.002)	0.045 (0.002)
	TARNet	0.111 (0.003)	0.082 (0.003)	0.038 (0.001)	0.031 (0.001)
	CFRNet	0.096 (0.003)	0.068 (0.002)	0.036 (0.001)	0.033 (0.001)
	DragonNet	0.100 (0.003)	0.071 (0.002)	0.034 (0.001)	0.027 (0.001)
	SNet	0.120 (0.004)	0.086 (0.003)	0.034 (0.001)	0.029 (0.001)
	ESCFR	0.128 (0.004)	0.096 (0.003)	0.040 (0.001)	0.033 (0.001)
	DIGNet	0.097 (0.003)	0.071 (0.002)	0.035 (0.001)	0.028 (0.001)
	TransTEE	0.134 (0.005)	0.092 (0.003)	0.050 (0.002)	0.040 (0.002)
	GRACE	0.077 (0.003)	0.053 (0.002)	0.026 (0.001)	0.019 (0.001)
	GRACE-R	0.076 (0.003)	0.052 (0.002)	0.024 (0.001)	0.018 (0.001)

I.3. (iii) Tree-based DGP

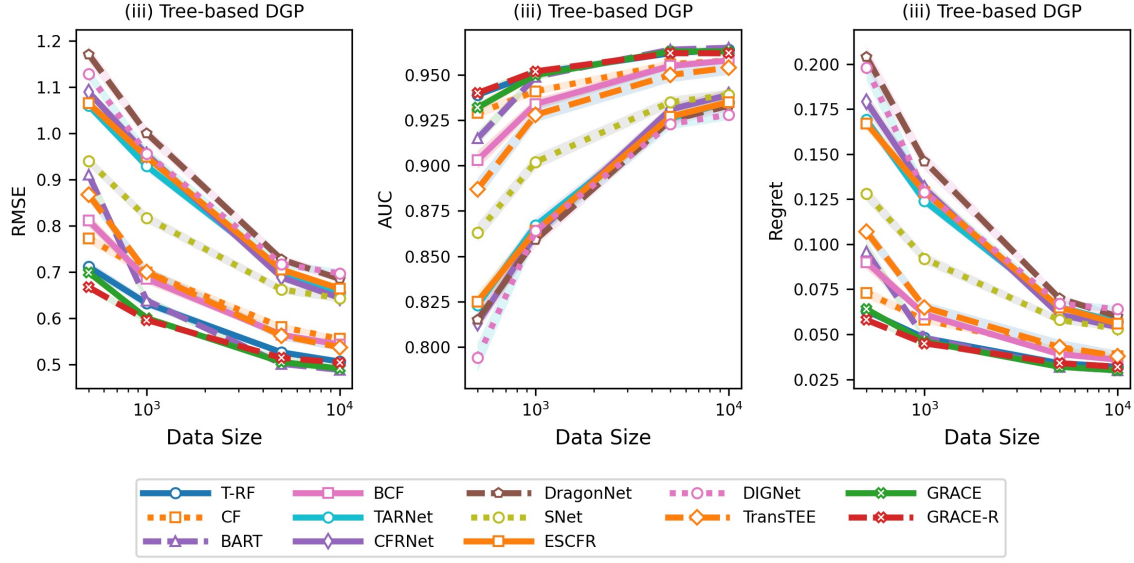


Figure 12. Performance comparison for (iii) Tree-based DGP. The shaded areas indicate standard error.

Figure 13. Full result for (iii) Tree-based DGP. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	10000
RMSE	T-RF	0.711 (0.012)	0.632 (0.010)	0.526 (0.006)	0.506 (0.005)
	CF	0.773 (0.011)	0.699 (0.010)	0.581 (0.007)	0.555 (0.006)
	BART	0.910 (0.020)	0.638 (0.010)	0.501 (0.005)	0.488 (0.005)
	BCF	0.811 (0.014)	0.685 (0.011)	0.564 (0.006)	0.543 (0.005)
	TARNet	1.060 (0.011)	0.929 (0.011)	0.702 (0.010)	0.656 (0.009)
	CFRNet	1.089 (0.011)	0.954 (0.012)	0.688 (0.009)	0.644 (0.008)
	DragonNet	1.171 (0.018)	1.000 (0.014)	0.727 (0.010)	0.685 (0.009)
	SNet	0.940 (0.011)	0.817 (0.011)	0.662 (0.009)	0.643 (0.009)
	ESCFR	1.065 (0.010)	0.949 (0.011)	0.705 (0.009)	0.664 (0.008)
	DIGNet	1.128 (0.014)	0.956 (0.012)	0.717 (0.008)	0.697 (0.009)
	TransTEE	0.867 (0.014)	0.700 (0.011)	0.562 (0.010)	0.537 (0.008)
	GRACE	0.699 (0.010)	0.600 (0.007)	0.505 (0.005)	0.491 (0.005)
	GRACE-R	0.667 (0.009)	0.596 (0.007)	0.514 (0.005)	0.504 (0.005)
AUC	T-RF	0.939 (0.002)	0.951 (0.002)	0.962 (0.001)	0.964 (0.001)
	CF	0.929 (0.003)	0.941 (0.002)	0.956 (0.002)	0.958 (0.001)
	BART	0.915 (0.004)	0.949 (0.002)	0.964 (0.001)	0.965 (0.001)
	BCF	0.903 (0.005)	0.934 (0.003)	0.955 (0.002)	0.958 (0.001)
	TARNet	0.823 (0.005)	0.867 (0.004)	0.926 (0.003)	0.935 (0.003)
	CFRNet	0.813 (0.005)	0.862 (0.004)	0.931 (0.003)	0.939 (0.003)
	DragonNet	0.815 (0.005)	0.859 (0.004)	0.925 (0.003)	0.933 (0.003)
	SNet	0.863 (0.005)	0.902 (0.004)	0.935 (0.003)	0.939 (0.003)
	ESCFR	0.825 (0.005)	0.864 (0.004)	0.927 (0.003)	0.935 (0.003)
	DIGNet	0.794 (0.008)	0.864 (0.005)	0.923 (0.003)	0.928 (0.003)
	TransTEE	0.887 (0.006)	0.928 (0.004)	0.950 (0.004)	0.954 (0.004)
	GRACE	0.932 (0.003)	0.950 (0.002)	0.963 (0.001)	0.963 (0.002)
	GRACE-R	0.940 (0.002)	0.952 (0.002)	0.962 (0.001)	0.962 (0.002)
Regret	T-RF	0.063 (0.002)	0.048 (0.001)	0.034 (0.001)	0.032 (0.001)
	CF	0.073 (0.003)	0.058 (0.002)	0.042 (0.001)	0.038 (0.001)
	BART	0.096 (0.006)	0.048 (0.001)	0.032 (0.001)	0.030 (0.001)
	BCF	0.090 (0.004)	0.061 (0.002)	0.039 (0.001)	0.036 (0.001)
	TARNet	0.169 (0.004)	0.124 (0.003)	0.065 (0.002)	0.057 (0.002)
	CFRNet	0.179 (0.005)	0.131 (0.004)	0.061 (0.002)	0.054 (0.001)
	DragonNet	0.204 (0.006)	0.146 (0.005)	0.070 (0.002)	0.060 (0.002)
	SNet	0.128 (0.004)	0.092 (0.003)	0.058 (0.002)	0.053 (0.001)
	ESCFR	0.167 (0.004)	0.129 (0.003)	0.065 (0.002)	0.056 (0.001)
	DIGNet	0.198 (0.007)	0.129 (0.004)	0.067 (0.002)	0.064 (0.002)
	TransTEE	0.107 (0.004)	0.065 (0.003)	0.043 (0.004)	0.038 (0.003)
	GRACE	0.064 (0.002)	0.046 (0.001)	0.032 (0.001)	0.030 (0.001)
	GRACE-R	0.058 (0.001)	0.045 (0.001)	0.034 (0.001)	0.032 (0.001)

I.4. (iv) Multi-valued Treatment Setting

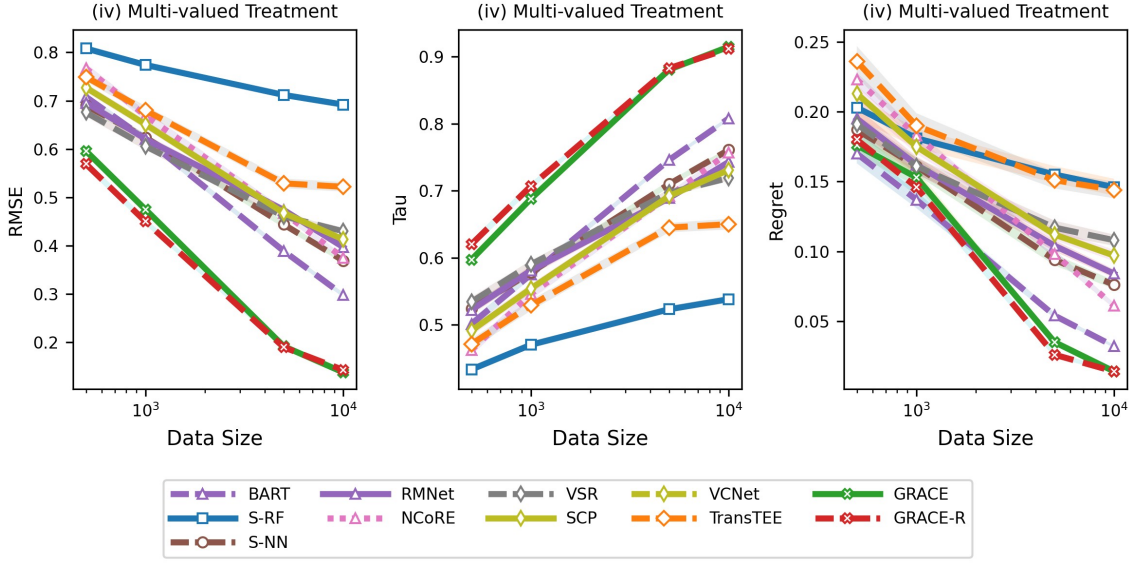

 Figure 14. Performance comparison for (iv) **Multi-valued Treatment Setting**. The shaded areas indicate standard error.

 Figure 15. Full result for (iv) **Multi-valued Treatment**. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	10000
RMSE	BART	0.708 (0.007)	0.621 (0.008)	0.389 (0.005)	0.298 (0.005)
	S-RF	0.808 (0.005)	0.774 (0.005)	0.712 (0.005)	0.692 (0.005)
	S-NN	0.690 (0.008)	0.624 (0.008)	0.444 (0.007)	0.369 (0.005)
	RMNet	0.696 (0.009)	0.621 (0.009)	0.474 (0.007)	0.397 (0.006)
	NCoRE	0.767 (0.010)	0.669 (0.010)	0.468 (0.009)	0.375 (0.005)
	VSR	0.676 (0.009)	0.606 (0.008)	0.461 (0.007)	0.430 (0.005)
	SCP	0.727 (0.009)	0.651 (0.009)	0.468 (0.008)	0.413 (0.005)
	TransTEE	0.749 (0.009)	0.680 (0.009)	0.529 (0.010)	0.522 (0.010)
	GRACE	0.596 (0.009)	0.475 (0.007)	0.192 (0.002)	0.138 (0.001)
	GRACE-R	0.569 (0.009)	0.450 (0.007)	0.190 (0.003)	0.143 (0.002)
Tau	BART	0.501 (0.007)	0.575 (0.006)	0.746 (0.003)	0.808 (0.003)
	S-RF	0.433 (0.005)	0.470 (0.005)	0.523 (0.005)	0.538 (0.005)
	S-NN	0.524 (0.007)	0.578 (0.006)	0.710 (0.005)	0.761 (0.003)
	RMNet	0.522 (0.007)	0.581 (0.007)	0.689 (0.005)	0.743 (0.004)
	NCoRE	0.462 (0.008)	0.546 (0.008)	0.692 (0.006)	0.757 (0.004)
	VSR	0.534 (0.007)	0.590 (0.006)	0.697 (0.005)	0.719 (0.004)
	SCP	0.491 (0.008)	0.554 (0.007)	0.692 (0.005)	0.731 (0.004)
	TransTEE	0.471 (0.008)	0.529 (0.008)	0.645 (0.007)	0.650 (0.008)
	GRACE	0.597 (0.007)	0.688 (0.005)	0.881 (0.002)	0.915 (0.001)
	GRACE-R	0.620 (0.007)	0.707 (0.005)	0.883 (0.002)	0.912 (0.001)
Regret	BART	0.170 (0.007)	0.137 (0.006)	0.054 (0.002)	0.032 (0.001)
	S-RF	0.203 (0.008)	0.181 (0.007)	0.155 (0.006)	0.146 (0.006)
	S-NN	0.187 (0.007)	0.161 (0.006)	0.094 (0.004)	0.076 (0.003)
	RMNet	0.195 (0.008)	0.162 (0.006)	0.104 (0.004)	0.084 (0.003)
	NCoRE	0.223 (0.005)	0.183 (0.004)	0.098 (0.003)	0.061 (0.002)
	VSR	0.191 (0.007)	0.161 (0.006)	0.117 (0.005)	0.108 (0.004)
	SCP	0.213 (0.006)	0.175 (0.005)	0.112 (0.004)	0.097 (0.004)
	TransTEE	0.236 (0.011)	0.190 (0.009)	0.151 (0.006)	0.144 (0.006)
	GRACE	0.176 (0.007)	0.153 (0.006)	0.035 (0.002)	0.014 (0.000)
	GRACE-R	0.180 (0.007)	0.146 (0.006)	0.026 (0.002)	0.014 (0.001)

I.5. (v) Continuous Treatment Setting

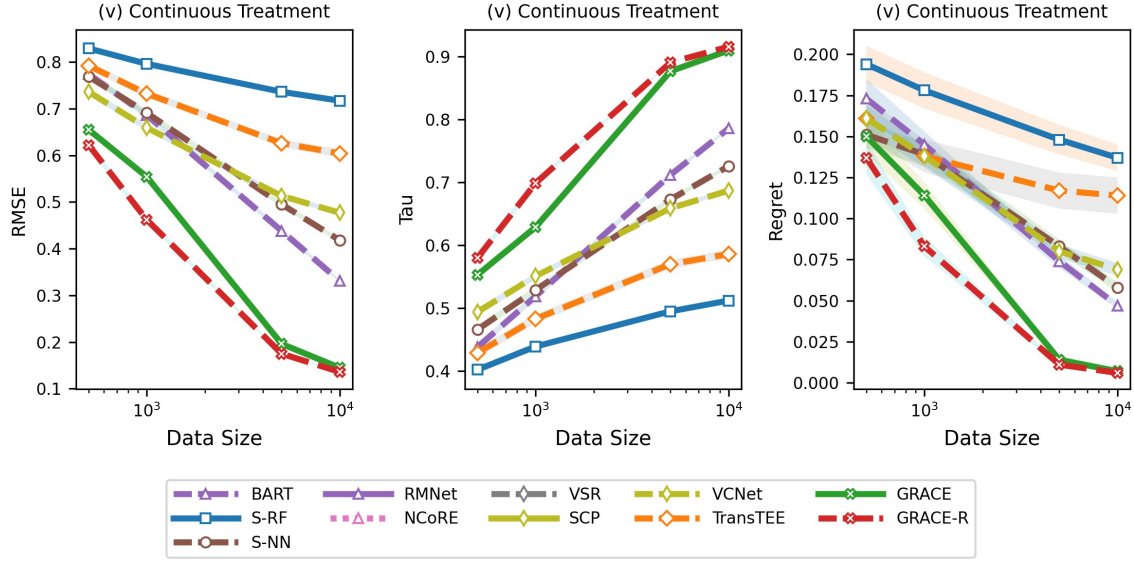


Figure 16. Performance comparison for (v) Continuous Treatment Setting. The shaded areas indicate standard error.

Figure 17. Full result for (v) Continuous Treatment. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	10000
RMSE	BART	0.774 (0.007)	0.686 (0.007)	0.438 (0.005)	0.331 (0.003)
	S-RF	0.829 (0.005)	0.796 (0.005)	0.736 (0.005)	0.717 (0.005)
	S-NN	0.769 (0.008)	0.691 (0.007)	0.495 (0.006)	0.418 (0.005)
	VCNet	0.736 (0.008)	0.659 (0.007)	0.513 (0.006)	0.477 (0.005)
	TransTEE	0.792 (0.008)	0.732 (0.008)	0.626 (0.008)	0.604 (0.009)
	GRACE	0.655 (0.008)	0.554 (0.007)	0.196 (0.002)	0.145 (0.001)
	GRACE-R	0.622 (0.008)	0.462 (0.007)	0.175 (0.002)	0.136 (0.002)
Tau	BART	0.438 (0.006)	0.519 (0.006)	0.712 (0.004)	0.786 (0.002)
	S-RF	0.402 (0.005)	0.439 (0.005)	0.495 (0.004)	0.512 (0.004)
	S-NN	0.466 (0.006)	0.529 (0.006)	0.673 (0.004)	0.726 (0.003)
	VCNet	0.494 (0.006)	0.551 (0.005)	0.659 (0.004)	0.687 (0.003)
	TransTEE	0.429 (0.008)	0.483 (0.007)	0.570 (0.007)	0.586 (0.007)
	GRACE	0.553 (0.006)	0.629 (0.006)	0.877 (0.001)	0.910 (0.001)
	GRACE-R	0.580 (0.006)	0.699 (0.005)	0.891 (0.001)	0.916 (0.001)
Regret	BART	0.173 (0.012)	0.145 (0.008)	0.074 (0.004)	0.047 (0.002)
	S-RF	0.194 (0.011)	0.178 (0.011)	0.148 (0.009)	0.137 (0.008)
	S-NN	0.151 (0.008)	0.139 (0.007)	0.083 (0.004)	0.058 (0.003)
	VCNet	0.161 (0.008)	0.138 (0.007)	0.080 (0.004)	0.069 (0.004)
	TransTEE	0.161 (0.012)	0.138 (0.010)	0.117 (0.011)	0.114 (0.011)
	GRACE	0.150 (0.011)	0.114 (0.008)	0.014 (0.001)	0.007 (0.000)
	GRACE-R	0.137 (0.010)	0.083 (0.006)	0.011 (0.001)	0.006 (0.000)

I.6. (vi) Bundle Treatment Setting

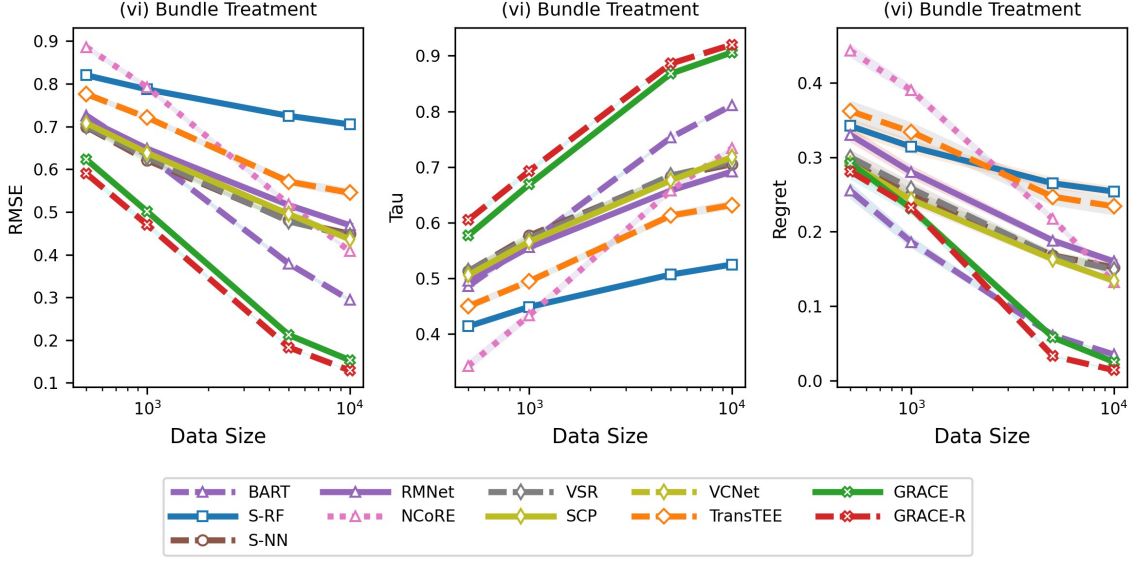

 Figure 18. Performance comparison for (vi) **Bundle Treatment Setting**. The shaded areas indicate standard error.

 Figure 19. Full result for (vi) **Bundle Treatment**. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	10000
RMSE	BART	0.726 (0.006)	0.633 (0.006)	0.379 (0.005)	0.294 (0.004)
	S-RF	0.820 (0.004)	0.787 (0.004)	0.725 (0.004)	0.705 (0.004)
	S-NN	0.699 (0.006)	0.621 (0.006)	0.480 (0.005)	0.449 (0.005)
	RMNet	0.718 (0.006)	0.648 (0.007)	0.516 (0.006)	0.469 (0.005)
	NCoRE	0.886 (0.007)	0.792 (0.007)	0.518 (0.007)	0.408 (0.005)
	VSR	0.701 (0.006)	0.628 (0.006)	0.478 (0.005)	0.444 (0.005)
	SCP	0.708 (0.006)	0.638 (0.006)	0.495 (0.006)	0.436 (0.004)
	TransTEE	0.776 (0.007)	0.721 (0.007)	0.571 (0.009)	0.545 (0.011)
	GRACE	0.623 (0.007)	0.501 (0.007)	0.212 (0.002)	0.153 (0.001)
	GRACE-R	0.590 (0.007)	0.470 (0.007)	0.183 (0.002)	0.129 (0.001)
Tau	BART	0.486 (0.006)	0.564 (0.006)	0.753 (0.003)	0.811 (0.002)
	S-RF	0.414 (0.005)	0.449 (0.004)	0.507 (0.004)	0.525 (0.004)
	S-NN	0.513 (0.006)	0.577 (0.005)	0.683 (0.004)	0.705 (0.003)
	RMNet	0.496 (0.006)	0.556 (0.006)	0.658 (0.005)	0.692 (0.004)
	NCoRE	0.343 (0.009)	0.434 (0.008)	0.658 (0.006)	0.734 (0.004)
	VSR	0.513 (0.006)	0.572 (0.005)	0.685 (0.004)	0.709 (0.003)
	SCP	0.506 (0.006)	0.566 (0.005)	0.676 (0.004)	0.718 (0.003)
	TransTEE	0.450 (0.007)	0.495 (0.006)	0.613 (0.007)	0.632 (0.008)
	GRACE	0.577 (0.006)	0.669 (0.005)	0.868 (0.002)	0.906 (0.001)
	GRACE-R	0.605 (0.006)	0.693 (0.005)	0.886 (0.001)	0.920 (0.001)
Regret	BART	0.255 (0.011)	0.186 (0.008)	0.061 (0.002)	0.035 (0.001)
	S-RF	0.342 (0.010)	0.314 (0.009)	0.265 (0.008)	0.254 (0.007)
	S-NN	0.293 (0.010)	0.250 (0.008)	0.168 (0.006)	0.152 (0.005)
	RMNet	0.330 (0.010)	0.280 (0.009)	0.188 (0.005)	0.160 (0.005)
	NCoRE	0.443 (0.009)	0.390 (0.007)	0.217 (0.005)	0.132 (0.004)
	VSR	0.300 (0.010)	0.258 (0.009)	0.167 (0.006)	0.149 (0.005)
	SCP	0.291 (0.010)	0.243 (0.008)	0.163 (0.005)	0.134 (0.005)
	TransTEE	0.362 (0.014)	0.334 (0.013)	0.247 (0.012)	0.234 (0.012)
	GRACE	0.292 (0.011)	0.231 (0.008)	0.058 (0.003)	0.025 (0.001)
	GRACE-R	0.281 (0.011)	0.232 (0.009)	0.033 (0.002)	0.014 (0.001)

I.7. IHDP Dataset

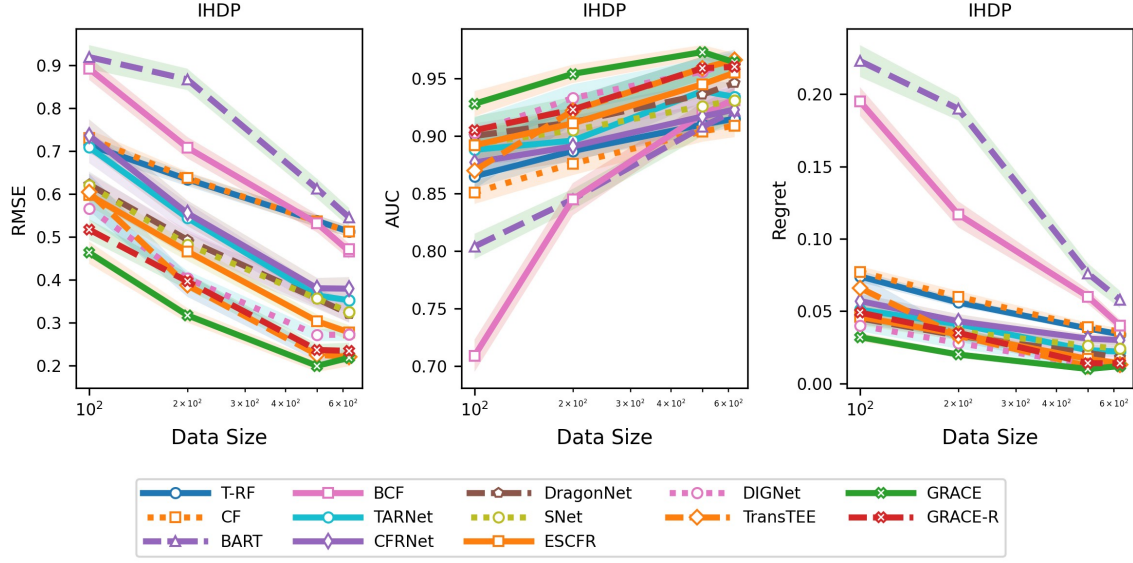

 Figure 20. Performance comparison for **IHDP** dataset. The shaded areas indicate standard error.

 Figure 21. Full result for **IHDP**. Standard errors are reported in parentheses.

Metric	Algorithm	100	200	500	627
RMSE	T-RF	0.714 (0.020)	0.634 (0.016)	0.537 (0.014)	0.512 (0.013)
	CF	0.730 (0.020)	0.637 (0.015)	0.536 (0.014)	0.512 (0.013)
	BART	0.919 (0.028)	0.867 (0.025)	0.612 (0.023)	0.545 (0.022)
	BCF	0.892 (0.026)	0.708 (0.025)	0.531 (0.021)	0.466 (0.018)
	TARNet	0.709 (0.037)	0.544 (0.035)	0.363 (0.024)	0.352 (0.027)
	CFRNet	0.736 (0.038)	0.555 (0.030)	0.380 (0.024)	0.379 (0.027)
	DragonNet	0.622 (0.026)	0.493 (0.023)	0.355 (0.020)	0.320 (0.020)
	SNet	0.620 (0.032)	0.482 (0.028)	0.356 (0.024)	0.325 (0.025)
	ESCFR	0.598 (0.027)	0.466 (0.022)	0.303 (0.016)	0.277 (0.017)
	DIGNet	0.566 (0.026)	0.403 (0.018)	0.271 (0.014)	0.272 (0.015)
	TransTEE	0.604 (0.043)	0.387 (0.027)	0.225 (0.015)	0.220 (0.016)
	GRACE	0.463 (0.026)	0.317 (0.017)	0.199 (0.014)	0.218 (0.020)
	GRACE-R	0.517 (0.024)	0.396 (0.019)	0.236 (0.014)	0.234 (0.019)
AUC	T-RF	0.865 (0.010)	0.887 (0.008)	0.910 (0.008)	0.915 (0.007)
	CF	0.851 (0.010)	0.876 (0.010)	0.904 (0.009)	0.909 (0.010)
	BART	0.804 (0.011)	0.845 (0.009)	0.908 (0.008)	0.922 (0.009)
	BCF	0.709 (0.014)	0.845 (0.014)	0.919 (0.010)	0.932 (0.010)
	TARNet	0.888 (0.013)	0.896 (0.014)	0.939 (0.010)	0.934 (0.012)
	CFRNet	0.877 (0.012)	0.891 (0.013)	0.917 (0.011)	0.923 (0.011)
	DragonNet	0.900 (0.011)	0.912 (0.012)	0.936 (0.010)	0.946 (0.009)
	SNet	0.893 (0.013)	0.905 (0.013)	0.926 (0.011)	0.931 (0.012)
	ESCFR	0.892 (0.014)	0.911 (0.014)	0.945 (0.011)	0.955 (0.009)
	DIGNet	0.903 (0.013)	0.933 (0.012)	0.956 (0.011)	0.964 (0.009)
	TransTEE	0.870 (0.016)	0.922 (0.012)	0.959 (0.010)	0.966 (0.009)
	GRACE	0.928 (0.011)	0.954 (0.008)	0.973 (0.006)	0.964 (0.008)
	GRACE-R	0.905 (0.012)	0.923 (0.011)	0.959 (0.010)	0.960 (0.008)
Regret	T-RF	0.074 (0.004)	0.056 (0.003)	0.038 (0.002)	0.034 (0.002)
	CF	0.077 (0.004)	0.060 (0.004)	0.039 (0.003)	0.036 (0.002)
	BART	0.223 (0.011)	0.190 (0.008)	0.076 (0.007)	0.058 (0.006)
	BCF	0.195 (0.010)	0.117 (0.009)	0.060 (0.005)	0.040 (0.004)
	TARNet	0.051 (0.005)	0.041 (0.005)	0.023 (0.004)	0.022 (0.004)
	CFRNet	0.057 (0.005)	0.043 (0.005)	0.031 (0.003)	0.030 (0.004)
	DragonNet	0.045 (0.004)	0.033 (0.003)	0.021 (0.003)	0.018 (0.003)
	SNet	0.046 (0.005)	0.036 (0.004)	0.026 (0.004)	0.024 (0.004)
	ESCFR	0.046 (0.006)	0.034 (0.004)	0.017 (0.002)	0.014 (0.002)
	DIGNet	0.040 (0.004)	0.028 (0.004)	0.013 (0.002)	0.012 (0.002)
	TransTEE	0.066 (0.009)	0.033 (0.004)	0.012 (0.002)	0.013 (0.002)
	GRACE	0.032 (0.003)	0.020 (0.002)	0.010 (0.002)	0.012 (0.002)
	GRACE-R	0.049 (0.006)	0.035 (0.004)	0.014 (0.002)	0.014 (0.002)

I.8. ACIC 2016 Dataset

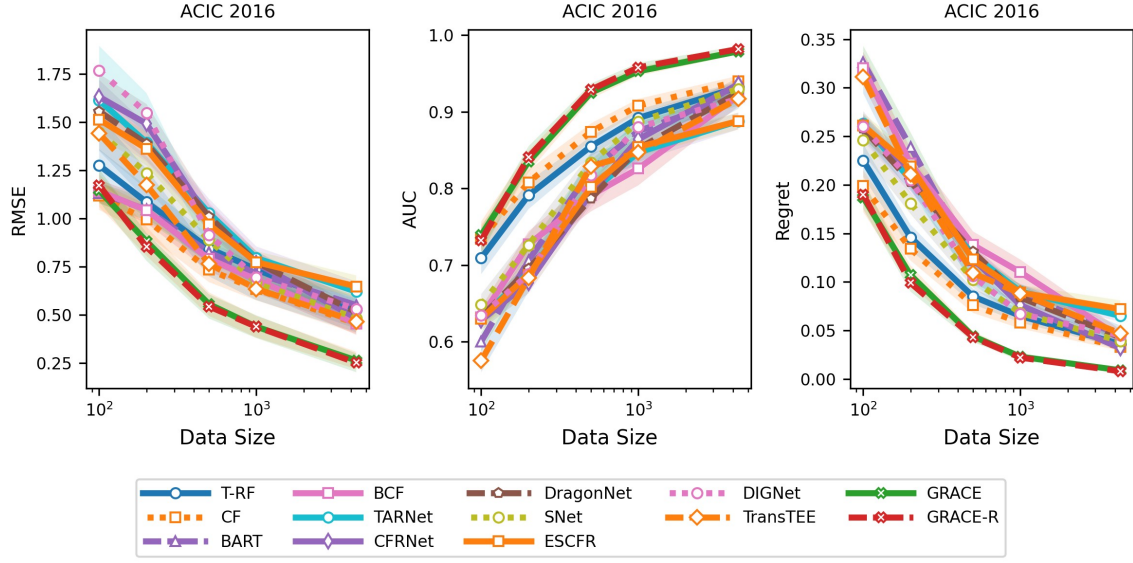

 Figure 22. Performance comparison for **ACIC 2016** dataset. The shaded areas indicate standard error.

 Figure 23. Full result for **ACIC 2016**. Standard errors are reported in parentheses.

Metric	Algorithm	100	200	500	1000	4321
RMSE	T-RF	1.275 (0.084)	1.084 (0.083)	0.844 (0.071)	0.735 (0.065)	0.528 (0.058)
	CF	1.117 (0.073)	0.993 (0.077)	0.733 (0.064)	0.632 (0.057)	0.455 (0.053)
	BART	1.131 (0.054)	1.046 (0.057)	0.824 (0.055)	0.714 (0.052)	0.550 (0.050)
	BCF	1.141 (0.059)	1.043 (0.068)	0.789 (0.064)	0.683 (0.062)	0.454 (0.058)
	TARNet	1.612 (0.106)	1.394 (0.093)	1.027 (0.074)	0.794 (0.060)	0.620 (0.053)
	CFRNet	1.630 (0.119)	1.491 (0.102)	0.997 (0.075)	0.767 (0.063)	0.483 (0.054)
	DragonNet	1.555 (0.104)	1.387 (0.093)	1.010 (0.073)	0.772 (0.059)	0.514 (0.054)
	SNet	1.447 (0.093)	1.235 (0.085)	0.887 (0.070)	0.669 (0.059)	0.482 (0.055)
	ESCFR	1.512 (0.094)	1.361 (0.086)	0.968 (0.070)	0.773 (0.057)	0.647 (0.057)
	DIGNet	1.767 (0.127)	1.546 (0.104)	0.914 (0.072)	0.693 (0.058)	0.530 (0.052)
	TransTEE	1.440 (0.090)	1.174 (0.081)	0.764 (0.065)	0.634 (0.058)	0.465 (0.056)
	GRACE	1.145 (0.086)	0.881 (0.075)	0.555 (0.061)	0.438 (0.056)	0.263 (0.045)
	GRACE-R	1.171 (0.084)	0.852 (0.072)	0.543 (0.063)	0.438 (0.059)	0.251 (0.049)
AUC	T-RF	0.709 (0.021)	0.791 (0.018)	0.855 (0.013)	0.892 (0.011)	0.932 (0.008)
	CF	0.734 (0.019)	0.808 (0.016)	0.874 (0.011)	0.908 (0.009)	0.940 (0.007)
	BART	0.600 (0.012)	0.706 (0.017)	0.824 (0.014)	0.875 (0.011)	0.920 (0.008)
	BCF	0.632 (0.013)	0.727 (0.017)	0.791 (0.021)	0.826 (0.022)	0.921 (0.014)
	TARNet	0.631 (0.012)	0.690 (0.013)	0.795 (0.012)	0.847 (0.012)	0.888 (0.011)
	CFRNet	0.628 (0.013)	0.677 (0.014)	0.794 (0.014)	0.864 (0.012)	0.936 (0.009)
	DragonNet	0.632 (0.013)	0.695 (0.013)	0.787 (0.013)	0.852 (0.012)	0.925 (0.009)
	SNet	0.648 (0.014)	0.726 (0.015)	0.834 (0.013)	0.887 (0.011)	0.930 (0.009)
	ESCFR	0.630 (0.013)	0.687 (0.015)	0.802 (0.012)	0.854 (0.012)	0.888 (0.011)
	DIGNet	0.634 (0.012)	0.688 (0.014)	0.817 (0.013)	0.880 (0.011)	0.917 (0.009)
	TransTEE	0.575 (0.015)	0.683 (0.018)	0.829 (0.017)	0.847 (0.017)	0.917 (0.013)
	GRACE	0.739 (0.018)	0.835 (0.014)	0.925 (0.008)	0.953 (0.006)	0.979 (0.004)
	GRACE-R	0.732 (0.017)	0.841 (0.014)	0.929 (0.008)	0.958 (0.006)	0.982 (0.004)
Regret	T-RF	0.225 (0.017)	0.146 (0.013)	0.085 (0.008)	0.065 (0.007)	0.036 (0.004)
	CF	0.199 (0.016)	0.134 (0.012)	0.076 (0.008)	0.058 (0.006)	0.033 (0.004)
	BART	0.326 (0.017)	0.239 (0.015)	0.122 (0.010)	0.088 (0.009)	0.045 (0.004)
	BCF	0.320 (0.017)	0.222 (0.015)	0.138 (0.014)	0.110 (0.013)	0.043 (0.008)
	TARNet	0.263 (0.013)	0.205 (0.011)	0.128 (0.008)	0.090 (0.007)	0.065 (0.006)
	CFRNet	0.260 (0.012)	0.215 (0.012)	0.124 (0.009)	0.076 (0.007)	0.032 (0.004)
	DragonNet	0.258 (0.013)	0.203 (0.012)	0.131 (0.009)	0.084 (0.006)	0.043 (0.005)
	SNet	0.246 (0.014)	0.181 (0.011)	0.102 (0.008)	0.068 (0.007)	0.039 (0.005)
	ESCFR	0.261 (0.013)	0.219 (0.014)	0.123 (0.008)	0.089 (0.008)	0.072 (0.009)
	DIGNet	0.260 (0.013)	0.205 (0.011)	0.106 (0.008)	0.067 (0.006)	0.045 (0.004)
	TransTEE	0.311 (0.017)	0.211 (0.015)	0.109 (0.011)	0.088 (0.010)	0.047 (0.008)
	GRACE	0.187 (0.015)	0.107 (0.010)	0.044 (0.005)	0.023 (0.003)	0.009 (0.002)
	GRACE-R	0.190 (0.015)	0.099 (0.009)	0.043 (0.005)	0.022 (0.003)	0.008 (0.001)

I.9. TCGA Dataset

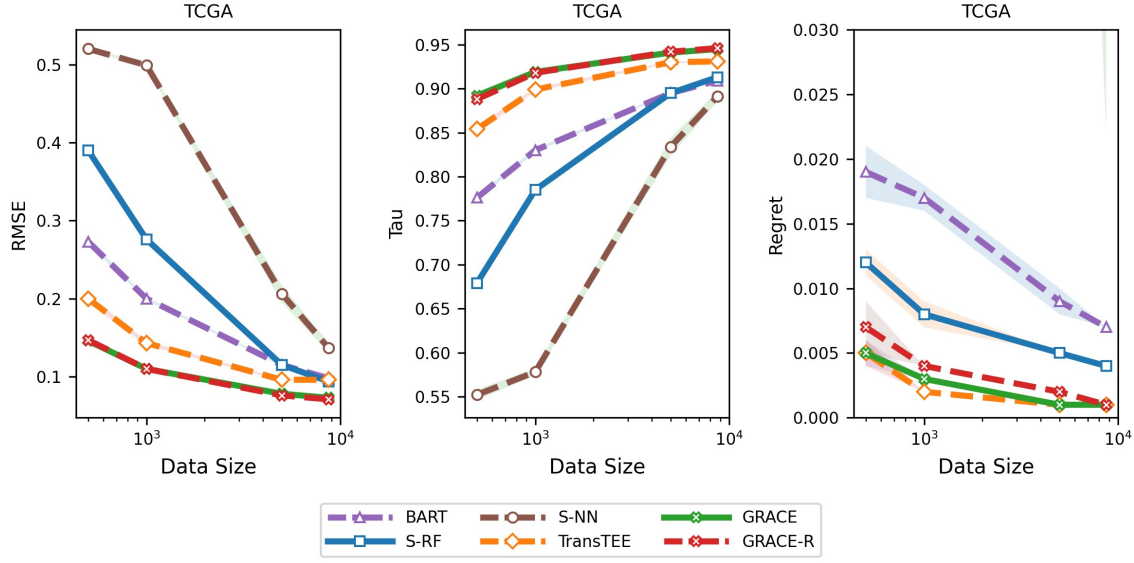

 Figure 24. Performance comparison for **TCGA** dataset. The shaded areas indicate standard error.

 Figure 25. Full result for **TCGA**. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	8693
RMSE	BART	0.273 (0.003)	0.200 (0.002)	0.115 (0.001)	0.098 (0.001)
	S-RF	0.390 (0.002)	0.276 (0.002)	0.115 (0.001)	0.094 (0.000)
	S-NN	0.520 (0.002)	0.499 (0.002)	0.206 (0.011)	0.137 (0.006)
	TransTEE	0.200 (0.005)	0.143 (0.003)	0.096 (0.001)	0.096 (0.001)
	GRACE	0.146 (0.003)	0.110 (0.002)	0.078 (0.001)	0.073 (0.001)
	GRACE-R	0.147 (0.003)	0.110 (0.002)	0.076 (0.001)	0.071 (0.001)
Tau	BART	0.776 (0.003)	0.830 (0.002)	0.895 (0.001)	0.909 (0.001)
	S-RF	0.679 (0.006)	0.785 (0.004)	0.895 (0.001)	0.913 (0.001)
	S-NN	0.552 (0.006)	0.578 (0.004)	0.834 (0.010)	0.891 (0.006)
	TransTEE	0.854 (0.004)	0.899 (0.002)	0.930 (0.001)	0.931 (0.001)
	GRACE	0.892 (0.003)	0.919 (0.002)	0.941 (0.001)	0.945 (0.001)
	GRACE-R	0.888 (0.003)	0.918 (0.001)	0.942 (0.001)	0.946 (0.001)
Regret	BART	0.019 (0.002)	0.017 (0.001)	0.009 (0.001)	0.007 (0.000)
	S-RF	0.012 (0.001)	0.008 (0.001)	0.005 (0.000)	0.004 (0.000)
	S-NN	1.156 (0.007)	1.149 (0.012)	0.147 (0.037)	0.042 (0.020)
	TransTEE	0.005 (0.001)	0.002 (0.000)	0.001 (0.000)	0.001 (0.000)
	GRACE	0.005 (0.001)	0.003 (0.000)	0.001 (0.000)	0.001 (0.000)
	GRACE-R	0.007 (0.002)	0.004 (0.000)	0.002 (0.000)	0.001 (0.000)

I.10. News Dataset

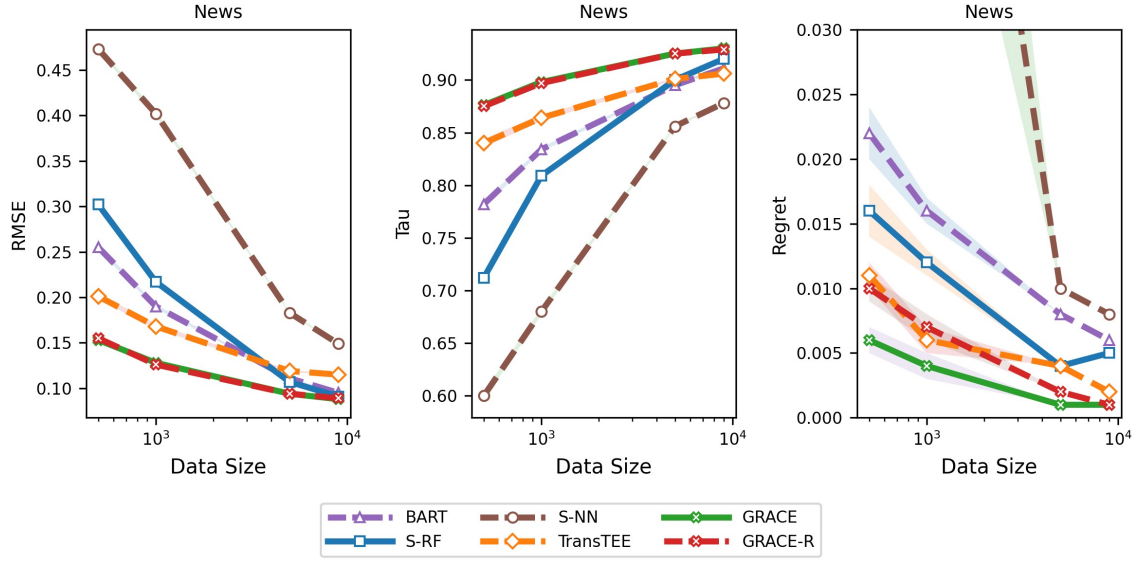

 Figure 26. Performance comparison for **News** dataset. The shaded areas indicate standard error.

 Figure 27. Full result for **News**. Standard errors are reported in parentheses.

Metric	Algorithm	500	1000	5000	9000
RMSE	BART	0.255 (0.003)	0.190 (0.002)	0.111 (0.001)	0.095 (0.000)
	S-RF	0.302 (0.002)	0.217 (0.002)	0.107 (0.001)	0.091 (0.000)
	S-NN	0.473 (0.002)	0.402 (0.003)	0.183 (0.001)	0.149 (0.001)
	TransTEE	0.201 (0.003)	0.168 (0.003)	0.119 (0.001)	0.115 (0.001)
	GRACE	0.153 (0.002)	0.128 (0.002)	0.094 (0.001)	0.088 (0.001)
	GRACE-R	0.155 (0.002)	0.126 (0.002)	0.094 (0.001)	0.089 (0.001)
Tau	BART	0.782 (0.003)	0.834 (0.002)	0.895 (0.001)	0.911 (0.001)
	S-RF	0.712 (0.005)	0.809 (0.003)	0.900 (0.001)	0.920 (0.001)
	S-NN	0.600 (0.004)	0.680 (0.003)	0.856 (0.001)	0.878 (0.001)
	TransTEE	0.840 (0.004)	0.864 (0.003)	0.901 (0.001)	0.906 (0.001)
	GRACE	0.876 (0.003)	0.898 (0.002)	0.925 (0.001)	0.930 (0.001)
	GRACE-R	0.875 (0.003)	0.897 (0.002)	0.925 (0.001)	0.929 (0.001)
Regret	BART	0.022 (0.002)	0.016 (0.001)	0.008 (0.000)	0.006 (0.000)
	S-RF	0.016 (0.002)	0.012 (0.001)	0.004 (0.000)	0.005 (0.000)
	S-NN	0.746 (0.047)	0.078 (0.023)	0.010 (0.000)	0.008 (0.000)
	TransTEE	0.011 (0.001)	0.006 (0.001)	0.004 (0.000)	0.002 (0.000)
	GRACE	0.006 (0.001)	0.004 (0.001)	0.001 (0.000)	0.001 (0.000)
	GRACE-R	0.010 (0.001)	0.007 (0.001)	0.002 (0.000)	0.001 (0.000)

I.11. Glucose (Binary) Dataset

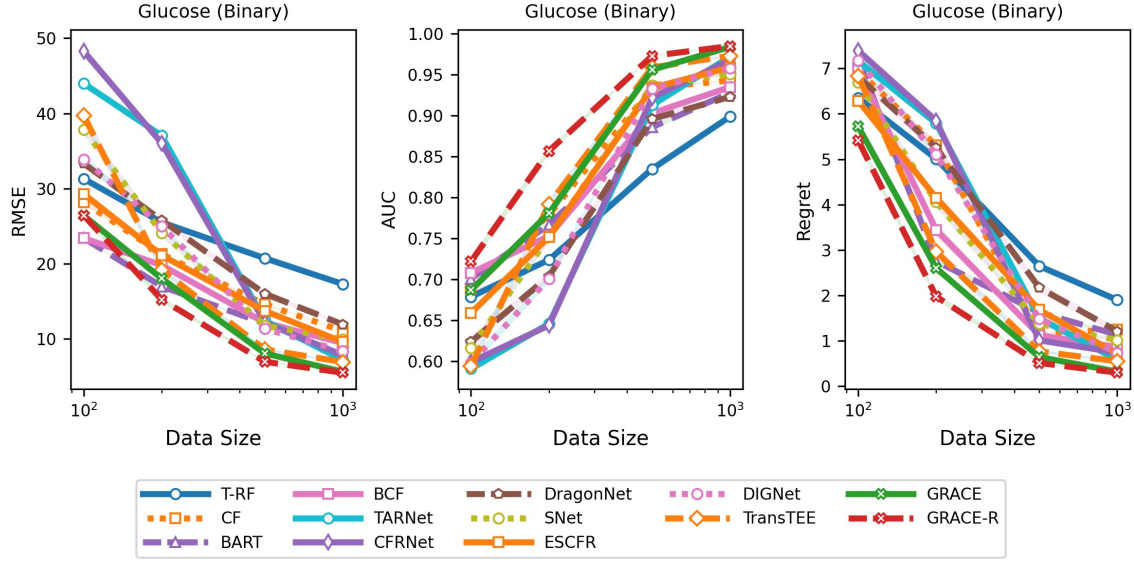

 Figure 28. Performance comparison for **Glucose (Binary)** dataset. The shaded areas indicate standard error.

 Figure 29. Full result for **Glucose (Binary)**. Standard errors are reported in parentheses.

Metric	Algorithm	100	200	500	1000
RMSE	T-RF	31.306 (0.392)	25.516 (0.223)	20.742 (0.139)	17.315 (0.145)
	CF	28.212 (0.400)	21.285 (0.203)	14.610 (0.143)	11.068 (0.077)
	BART	23.428 (0.102)	16.990 (0.123)	12.060 (0.111)	9.543 (0.078)
	BCF	23.419 (0.088)	19.808 (0.123)	12.184 (0.086)	9.454 (0.075)
	TARNet	43.985 (0.518)	37.035 (0.523)	12.436 (0.569)	7.224 (0.059)
	CFRNet	48.263 (0.698)	36.028 (0.454)	12.196 (0.477)	7.837 (0.120)
	DragonNet	33.343 (0.549)	25.787 (0.474)	16.016 (0.419)	11.941 (0.112)
	SNet	37.826 (0.653)	24.130 (0.587)	11.775 (0.172)	9.983 (0.116)
	ESCFR	29.279 (0.365)	21.121 (0.312)	13.732 (0.125)	9.614 (0.094)
	DIGNet	33.907 (0.616)	24.977 (0.435)	11.359 (0.131)	8.388 (0.078)
	TransTEE	39.717 (0.918)	19.172 (0.326)	8.627 (0.155)	6.944 (0.083)
	GRACE	26.413 (0.415)	18.039 (0.280)	8.087 (0.169)	5.646 (0.041)
	GRACE-R	26.434 (0.475)	15.246 (0.367)	6.974 (0.092)	5.524 (0.024)
AUC	T-RF	0.679 (0.006)	0.724 (0.003)	0.835 (0.003)	0.899 (0.001)
	CF	0.688 (0.003)	0.779 (0.003)	0.934 (0.001)	0.943 (0.001)
	BART	0.701 (0.005)	0.767 (0.004)	0.886 (0.003)	0.928 (0.002)
	BCF	0.708 (0.004)	0.753 (0.002)	0.903 (0.002)	0.935 (0.001)
	TARNet	0.591 (0.005)	0.646 (0.004)	0.913 (0.006)	0.972 (0.001)
	CFRNet	0.599 (0.005)	0.644 (0.004)	0.922 (0.004)	0.968 (0.001)
	DragonNet	0.624 (0.006)	0.704 (0.005)	0.896 (0.005)	0.923 (0.002)
	SNet	0.616 (0.006)	0.752 (0.006)	0.937 (0.002)	0.951 (0.001)
	ESCFR	0.659 (0.004)	0.752 (0.005)	0.934 (0.002)	0.959 (0.001)
	DIGNet	0.598 (0.006)	0.701 (0.005)	0.933 (0.002)	0.958 (0.001)
	TransTEE	0.594 (0.007)	0.792 (0.006)	0.959 (0.002)	0.973 (0.001)
	GRACE	0.687 (0.007)	0.781 (0.005)	0.956 (0.003)	0.984 (0.001)
	GRACE-R	0.722 (0.006)	0.857 (0.004)	0.973 (0.001)	0.985 (0.000)
Regret	T-RF	6.354 (0.045)	5.009 (0.113)	2.643 (0.041)	1.896 (0.025)
	CF	6.953 (0.041)	5.302 (0.103)	1.155 (0.031)	1.244 (0.017)
	BART	7.219 (0.182)	2.747 (0.070)	1.622 (0.049)	1.129 (0.026)
	BCF	6.980 (0.066)	3.441 (0.144)	1.118 (0.026)	0.857 (0.020)
	TARNet	7.170 (0.103)	5.792 (0.092)	1.495 (0.112)	0.572 (0.017)
	CFRNet	7.390 (0.104)	5.848 (0.096)	1.004 (0.031)	0.736 (0.026)
	DragonNet	6.820 (0.112)	5.251 (0.122)	2.166 (0.110)	1.205 (0.075)
	SNet	6.697 (0.105)	4.060 (0.125)	1.367 (0.050)	1.006 (0.034)
	ESCFR	6.290 (0.082)	4.148 (0.093)	1.679 (0.059)	0.708 (0.019)
	DIGNet	7.182 (0.108)	5.104 (0.110)	1.477 (0.044)	0.731 (0.025)
	TransTEE	6.838 (0.090)	2.962 (0.117)	0.762 (0.036)	0.545 (0.016)
	GRACE	5.722 (0.103)	2.598 (0.084)	0.637 (0.022)	0.316 (0.009)
	GRACE-R	5.417 (0.095)	1.971 (0.076)	0.506 (0.019)	0.294 (0.006)

I.12. Glucose (Non-binary) Dataset

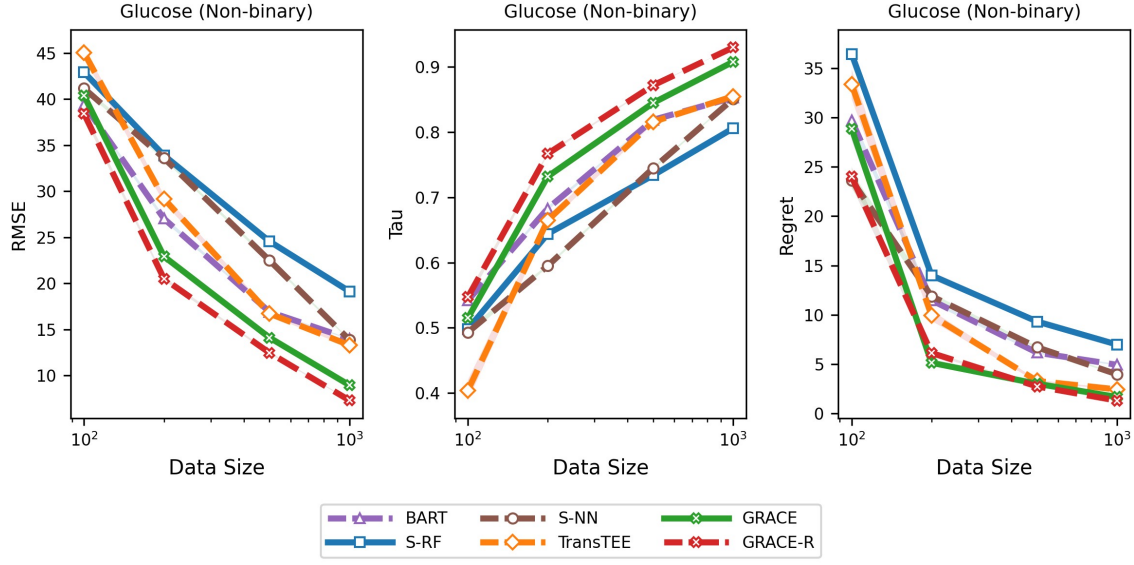


Figure 30. Performance comparison for **Glucose (Non-binary)** dataset. The shaded areas indicate standard error.

Figure 31. Full result for **Glucose (Non-binary)**. Standard errors are reported in parentheses.

Metric	Algorithm	100	200	500	1000
RMSE	BART	39.382 (0.202)	27.019 (0.264)	16.895 (0.079)	13.969 (0.073)
	S-RF	42.898 (0.112)	33.887 (0.144)	24.549 (0.079)	19.127 (0.058)
	S-NN	41.154 (0.277)	33.593 (0.167)	22.481 (0.182)	13.864 (0.088)
	TransTEE	45.032 (0.576)	29.176 (0.459)	16.705 (0.245)	13.287 (0.196)
	GRACE	40.371 (0.340)	22.915 (0.259)	14.100 (0.119)	8.944 (0.078)
	GRACE-R	38.405 (0.410)	20.497 (0.197)	12.451 (0.105)	7.294 (0.053)
Tau	BART	0.542 (0.002)	0.683 (0.003)	0.819 (0.001)	0.853 (0.001)
	S-RF	0.498 (0.002)	0.644 (0.002)	0.734 (0.001)	0.806 (0.001)
	S-NN	0.493 (0.005)	0.595 (0.002)	0.745 (0.002)	0.851 (0.001)
	TransTEE	0.404 (0.016)	0.665 (0.006)	0.816 (0.003)	0.855 (0.002)
	GRACE	0.515 (0.005)	0.732 (0.003)	0.845 (0.002)	0.908 (0.001)
	GRACE-R	0.547 (0.006)	0.767 (0.002)	0.872 (0.001)	0.930 (0.000)
Regret	BART	29.700 (0.586)	11.476 (0.323)	6.167 (0.101)	4.938 (0.081)
	S-RF	36.436 (0.655)	13.996 (0.395)	9.322 (0.166)	6.991 (0.091)
	S-NN	23.636 (0.840)	11.886 (0.298)	6.709 (0.118)	3.977 (0.076)
	TransTEE	33.361 (1.495)	9.923 (0.535)	3.300 (0.109)	2.448 (0.063)
	GRACE	28.857 (1.194)	5.140 (0.129)	3.018 (0.063)	1.674 (0.022)
	GRACE-R	23.998 (0.982)	6.131 (0.159)	2.747 (0.053)	1.309 (0.018)